

Meituan-Robotics-0: A Vision-Language-Action Foundation Model for Desktop Manipulation

LongCat Robotics Team

ABSTRACT

Contents

1	Introdcution	4
2	Data Curation	4
2.1	Vision-Language Data	4
2.2	Robotic Data	7
2.2.1	Data Overview	7
2.2.2	Data Curation Pipeline	8
3	Model Architecture	9
3.1	Problem Formulation	9
3.2	Vision-Language Backbone	10
3.3	Action Head	10
3.3.1	FAST: discrete action tokens	10
3.3.2	MMDiT with JiT parameterization	10
3.4	Multi-Embodiment Action Space	11
3.5	Language Conditioning	11
4	Training and Deployment	12
4.1	Pre-training	12
4.1.1	Recipe	12
4.1.2	Data	12
4.2	Mid-training	13
4.3	Real-World Post-Training	13
4.3.1	Supervised Fine-Tuning	13
4.3.2	HG-DAGger-Style Human Intervention	13
4.3.3	RECAP: Advantage-Conditioned Policy Learning	14
4.4	Deployment	16
4.4.1	Action Chunk Merging	16
5	Embodied Agent System	19
5.1	Event-Driven Task Planning and Execution	19
5.2	Monitor	19
5.3	Memory Manager	20
5.4	Planner	20
5.5	Asynchronous Execution Pipeline	21
6	Experiments	21
6.1	Simulation	21
6.1.1	RoboTwin 2.0	21
6.1.2	RoboDojo (Waiting for the best result.)	22

6.2	Real-Robot Evaluation	22
6.2.1	Evaluation Setup	22
6.2.2	Main VLA Comparison	24
6.2.3	Effect of HG-Dagger-Style Human Intervention	24
6.2.4	RECAP Evaluation	25
6.2.5	Action-Chunk Merging Evaluation	25
6.3	Embodied Agent System Behaviors	25
7	Future Work and Conclusion	27
8	Contributions	28
A	Real-Robot Setup and Data Preparation	29
B	RECAP Implementation Details	30



Figure 1

1 Introduction

2 Data Curation

2.1 Vision-Language Data

To strengthen visual-semantic understanding and embodied reasoning, we curate a large-scale vision-language dataset containing 147 million. As illustrated in Figure 2, 78% of the dataset consists of general vision-language data, while the remaining 22% is embodied vision-language data. The general portion covers general-purpose visual question answering, spatial understanding, and visual grounding and counting, whereas the embodied portion covers scene captioning, task planning and reasoning, visual grounding, and target-affordance grounding. Among these, target-affordance grounding jointly localizes the target object with a bounding box and marks an actionable point on the object to indicate where the robot should interact. For the embodied VQA portion, we augment the collected open-source datasets with a large volume of in-house samples derived from real-robot trajectories. To determine which capabilities should be prioritized in this data construction, we evaluate eight candidate VQA tasks grouped into three capability families: spatial localization (2D grounding, 2D pointing, and 3D grounding), action-related understanding (2D trajectory, 2D target-affordance grounding, and 3D target-affordance grounding), and semantic and task reasoning (subtask prediction and scene captioning). We then conduct controlled ablations on CALVIN using the StarVLA framework [37]. For all variants, the action policy is trained on environments A, B, and C using the same configuration and evaluated on environment D, while the corresponding VQA data are constructed from the action training set. Table 1 reports the success rates for completing at least k consecutive subtasks ($k = 1, \dots, 5$), together with the average number of consecutively completed subtasks per five-task sequence (Avg. Len.). Taking the best-performing task from each capability family, 3D grounding, 2D target-affordance grounding, and scene captioning achieve the highest Avg. Len. within spatial localization, action-related understanding, and semantic and task reasoning, respectively, with scores of 4.037, 4.102, and 4.082. However, 3D grounding requires additional 3D geometric annotations, making it difficult to scale reliably across large volumes of real-robot trajectories. We therefore adopt 2D grounding as a more scalable alternative for spatial localization. Although its Avg. Len. is lower at 3.941, it still increases the success rate for

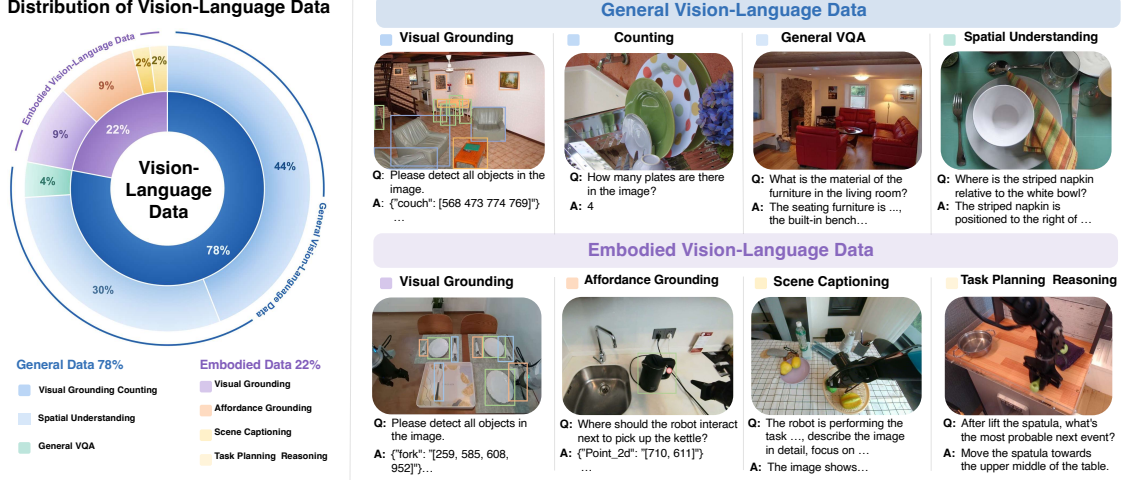


Figure 2: **Composition of the vision-language dataset.** The dataset contains 147 million samples, of which 78% are general vision-language data and the remaining 22% are embodied vision-language data. The general portion comprises general-purpose visual question answering (VQA), spatial understanding, visual grounding, and counting. The embodied portion comprises scene captioning, task planning and reasoning, visual grounding, and target-affordance grounding. Representative examples of the major data types are shown on the right. Need to be modified

completing all five consecutive subtasks from 62.8% to 67.8%. Considering downstream performance, complementary capability coverage, and data construction cost, we ultimately focus our in-house data expansion on 2D target-affordance grounding, scene captioning, and 2D grounding.

For our in-house 2D grounding data, we apply Set-of-Mark (SoM) prompting [46]. Specifically, we use the Segment Anything Model (SAM)[20] to segment and mark candidate regions, prompt a vision-language model to identify the object corresponding to each marked region, and convert the resulting object–region associations into grounding question–answer pairs. For 2D target-affordance grounding, we identify the key interaction frame of each action phase in successful episodes. We then propagate the target object’s bounding box and 2D interaction point to preceding frames within the same phase, using them as target-affordance supervision. For scene captioning, we prompt a vision-language model with robot observations and task context to describe task-relevant objects, spatial layouts, gripper states, and task progress.

Table 1: **Ablation of embodied vision-language supervision on CALVIN ABC→D.** Models are trained on CALVIN environments A, B, and C and evaluated on environment D. All variants use the same configuration. Columns Task 1–Task 5 report the success rates for completing at least k consecutive subtasks, where $k \in \{1, \dots, 5\}$. Avg. Len. denotes the average number of consecutively completed subtasks per five-task evaluation sequence.

Capability family	Training setting	Avg. Len.	Task 1	Task 2	Task 3	Task 4	Task 5
Baseline	–	3.832	91.0%	83.0%	76.2%	70.2%	62.8%
Spatial localization	+ 2D Grounding	3.941	91.3%	83.7%	78.1%	73.2%	67.8%
	+ 2D Pointing	3.806	90.8%	81.8%	74.8%	69.9%	63.3%
	+ 3D Grounding	4.037	92.4%	85.3%	79.6%	75.7%	70.7%
Action-related understanding	+ 2D Target-Affordance Grounding	4.102	93.3%	87.9%	82.5%	76.4%	70.1%
	+ 3D Target-Affordance Grounding	4.098	93.7%	89.9%	81.2%	73.9%	71.1%
	+ 2D Trajectory	3.987	92.0%	85.0%	79.2%	74.0%	68.5%
Semantic and task reasoning	+ Subtask Prediction	3.851	90.0%	82.2%	76.2%	71.7%	65.0%
	+ Scene Captioning	4.082	93.1%	87.1%	81.4%	76.2%	70.4%

2.2 Robotic Data

2.2.1 Data Overview

The final corpus contains 15,060 trajectory hours retained after curation. It combines in-house and open-source data from real and simulated environments and is restricted to stationary-base tabletop manipulation; segments containing robot-base motion are excluded. Unless otherwise noted, all shares in this subsection use the complete 15,060-hour corpus as their denominator.

Source composition. As shown in Figure 3, in-house real-world demonstrations account for 7,530 hours (50.0%), open-source real-world data for 3,235 hours (21.5%), and open-source simulation data for 4,295 hours (28.5%). The corpus therefore contains 10,765 hours of physical robot execution and 4,295 hours of simulation, while the in-house and open-source portions each contribute 7,530 hours.

Embodiment coverage. The corpus spans 12 robot embodiments. AgiBot G1 contributes 3,652 hours, followed by Agilex Aloha with 2,649 hours and Kuavo 4Pro with 2,444 hours. Together, these three embodiments provide 8,745 hours (58.1%); the other nine contribute 6,315 hours in total, with individual durations ranging from 112 to 1,580 hours. The in-house portion is distributed across seven configurations: AgiBot G1, Agilex Aloha, Kuavo 4Pro, Galaxea R1, WheelLoong M1, Qinglong, and Kuavo LB.

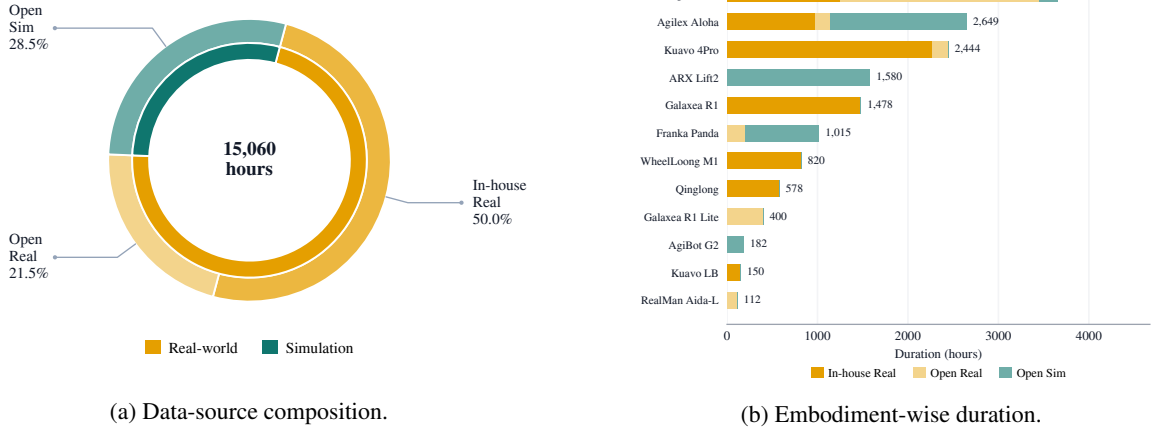


Figure 3: **Composition of the robotic manipulation corpus.** The corpus contains 15,060 retained trajectory hours across 12 robot embodiments. (a) Source composition; the inner ring aggregates real-world and simulation domains. (b) Embodiment-wise duration, with bar colors indicating the corresponding data source.

Open-source subset. Table 2 lists the nine open-source datasets retained after curation, comprising 3,235 hours of real-world trajectories and 4,295 hours of simulation.

Table 2: **Open-source robotic datasets retained after curation.** Hours and shares are measured against the complete 15,060-hour corpus.

Open real-world — 3,235 h (21.5%)			Open simulation — 4,295 h (28.5%)		
Data source	Hours	Share	Data source	Hours	Share
AgiBotWorld-Beta [1]	2,174	14.4%	InternData-A1 [40]	3,967	26.3%
Galaxea Open-World [16]	333	2.2%	Genie Sim 3.0 [49]	213	1.4%
RoboCOIN [43]	311	2.1%	RoboTwin 2.0 [29]	115	0.8%
LET [21]	172	1.1%			
DROID [19]	164	1.1%			
RoboMIND [42]	81	0.5%			
Open-source total: 7,530 h (50.0% of the full corpus)					

Within the 3,235-hour open-source real-world subset, AgiBotWorld-Beta is the largest contributor, providing 2,174 hours (67.2%). The other sources broaden embodiment coverage: LET and Galaxea Open-World provide bimanual humanoid or humanoid-style data, DROID provides single-arm Franka Panda data, and RoboCOIN and RoboMIND provide multi-embodiment data. Consistent with the corpus-wide scope, only stationary-base tabletop manipulation segments are retained from every source.

The open-source simulation subset is dominated by InternData-A1, which contributes 3,967 of 4,295 hours (92.4%) and covers single- and dual-arm pick-and-place, articulated-object interaction, and multi-stage tasks. Genie Sim 3.0 and RoboTwin 2.0 provide the remaining 328 hours of simulated tabletop trajectories.

2.2.2 Data Curation Pipeline

Raw trajectories are first processed by three filtering steps (S1–S3), which remove invalid boundary segments, inconsistent joint–EEF recordings, and visual corruption. The retained segments then pass through two alignment stages (A1–A2): A1 canonicalizes actions across robot embodiments, while A2 aligns multi-granularity task annotations with the retained trajectories. Figure 4 summarizes this two-part pipeline.

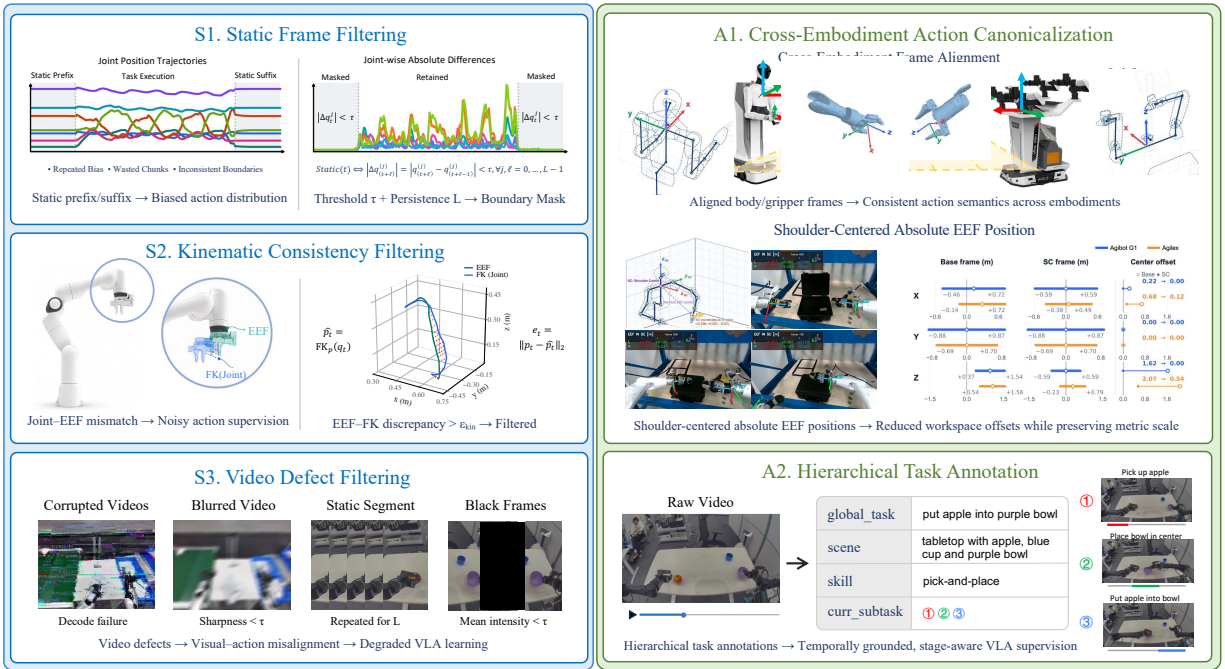


Figure 4: **Robot trajectory curation and alignment pipeline.** S1–S3 remove static boundaries, kinematic inconsistencies, and video defects; A1 canonicalizes actions across embodiments, and A2 aligns multi-granularity task annotations with the retained trajectories.

Static-frame filtering. Teleoperated episodes are recorded over a wider temporal window than the task itself. Recording begins before the operator starts manipulation and ends after task completion and post-task reset. Consequently, the recorded episode boundaries do not coincide with the actual task-execution interval, leaving stationary prefixes and suffixes with repeated near-zero actions. Let $\mathbf{q}_t \in \mathbb{R}^J$ be the recorded joint position and let $\Delta q_t^{(j)} = q_t^{(j)} - q_{t-1}^{(j)}$. We classify a time index as static only when every joint remains below a motion threshold for L consecutive steps:

$$\text{Static}(t) \iff |\Delta q_{t+\ell}^{(j)}| < \tau_{\text{stat}}, \quad \forall j \in \{1, \dots, J\}, \forall \ell \in \{0, \dots, L-1\}. \quad (1)$$

The persistence window prevents brief low-speed motions during task execution from being classified as idle. We trim only stationary segments connected to the beginning or end of an episode. The corresponding timestamps are removed jointly from the video, state, and action streams to preserve temporal alignment.

Kinematic-consistency filtering. For each embodiment, we reconstruct the end-effector (EEF) position from the recorded joint states using the kinematic chain specified in its URDF. After expressing the reconstructed and logged

EEF positions in the same source coordinate frame, we compare their discrepancy. Samples exceeding an embodiment-specific threshold are masked, and trajectories with persistent violations are discarded.

Video-defect filtering. We screen videos for decoding failures, blur, frozen segments, and black frames, removing corrupted spans from the synchronized trajectory streams and discarding episodes with persistent corruption. Detection thresholds are calibrated separately for each embodiment and data source.

Cross-embodiment action canonicalization. We define common right-handed body- and gripper-frame conventions across embodiments. The body axes point forward, left, and up $(+x, +y, +z)$. At the open-gripper zero pose, the fingers lie in the y - z plane: $+z$ points from the mounting flange to the gripper tip, $+y$ from the right finger to the left, and $+x$ completes the frame.

EEF poses are represented in a canonical shoulder-centered frame, denoted SC. For bimanual embodiments, SC is shared by both arms, with its origin at the shoulder center. For a single-arm embodiment without an explicit SC, the arm-root frame is designated as SC. In either case, SC uses the forward-left-up axis convention.

We index arms by i , using $i = 1$ for single-arm data and $i \in \{L, R\}$ for bimanual data, where L and R denote the left and right arms. At timestep t , ${}^{\text{SC}}\mathbf{p}_{\text{EEF},t}^{(i)} \in \mathbb{R}^3$ is the absolute EEF position target in SC, $\mathbf{r}_t^{(i)} \in \mathbb{R}^6$ is the continuous 6D representation of the EEF orientation in SC [53], and $g_t^{(i)} \in [0, 1]$ is the normalized gripper-closure fraction, where 0 denotes fully open and 1 fully closed. These components form a 10D per-arm action and, for bimanual data, a 20D action concatenated in left-right order:

$$\mathbf{a}_t^{(i)} = \begin{bmatrix} {}^{\text{SC}}\mathbf{p}_{\text{EEF},t}^{(i)} \\ \mathbf{r}_t^{(i)} \\ g_t^{(i)} \end{bmatrix} \in \mathbb{R}^{10}, \quad \mathbf{a}_t = \begin{bmatrix} \mathbf{a}_t^{(L)} \\ \mathbf{a}_t^{(R)} \end{bmatrix} \in \mathbb{R}^{20} \quad (\text{bimanual}). \quad (2)$$

Single-arm data use $\mathbf{a}_t = \mathbf{a}_t^{(1)} \in \mathbb{R}^{10}$.

Let B be the source body frame, ${}^{\text{SC}}\mathbf{T}_B$ the embodiment-specific 4×4 homogeneous rigid-body transform from B to SC, and ${}^B\mathbf{T}_{\text{EEF},t}^{(i)}$ the source EEF target pose expressed in B. The canonical pose is

$${}^{\text{SC}}\mathbf{T}_{\text{EEF},t}^{(i)} = {}^{\text{SC}}\mathbf{T}_B {}^B\mathbf{T}_{\text{EEF},t}^{(i)} = \begin{bmatrix} {}^{\text{SC}}\mathbf{R}_{\text{EEF},t}^{(i)} & {}^{\text{SC}}\mathbf{p}_{\text{EEF},t}^{(i)} \\ \mathbf{0}^\top & 1 \end{bmatrix}, \quad (3)$$

$$\mathbf{r}_t^{(i)} = \rho_{6D} \left({}^{\text{SC}}\mathbf{R}_{\text{EEF},t}^{(i)} \right).$$

The leading superscript denotes the reference frame; thus ${}^{\text{SC}}\mathbf{T}_{\text{EEF},t}^{(i)}$ is the EEF target pose expressed in SC. Its rotation and translation blocks are ${}^{\text{SC}}\mathbf{R}_{\text{EEF},t}^{(i)}$ and ${}^{\text{SC}}\mathbf{p}_{\text{EEF},t}^{(i)}$, respectively, and ρ_{6D} maps the rotation block to $\mathbf{r}_t^{(i)}$. The translation block therefore provides the metric absolute position target in SC, rather than a temporal displacement.

Hierarchical task annotation. The hierarchy links the episode-level goal in `global_task` to the active execution stage in `curr_subtask`. Each `curr_subtask` label is assigned to the interval in which that stage is performed. The auxiliary scene field describes the task-relevant objects and their spatial arrangement, while `skill` names the manipulation primitive. For example, an apple-in-bowl trajectory is partitioned into subtask intervals for picking up the apple, positioning the bowl, and placing the apple in the bowl.

The resulting corpus pairs synchronized observations and canonical EEF actions with episode-level goals, time-aligned subtask labels, and auxiliary scene and skill context.

3 Model Architecture

This section describes the model behind Meituan-Robotics-0. Our design follows the now-standard *VLM backbone + action head* recipe [6, 30, 33], instantiated with two choices tailored to desktop manipulation: a single Qwen3.5-4B backbone paired with two stage-dependent action heads (a discrete FAST [31] tokenizer for pre-training, a continuous MMDiT flow-matching head with JiT [24] parameterization for mid-training and downstream fine-tuning), and a unified 20-dimensional action space handled through padding and masking. Training recipes are deferred to Section 4.

3.1 Problem Formulation

A desktop VLA policy takes as input multi-view images $o_t = \{I_t^{(c)}\}_{c=1}^C$ from a small number of cameras (e.g., $C=3$: one head-mounted and two wrist-mounted), a natural-language instruction l describing the task, and, for the continuous

action head, the current robot proprioceptive state s_t . It produces an H -step chunk of future actions $a_{t:t+H-1}$. We model the continuous policy as

$$\pi_{\theta}(a_{t:t+H-1} \mid o_t, l, s_t), \quad (4)$$

shared across embodiments through the unified action-space interface described in Section 3.4. At each stage, the parameters θ cover the VLM backbone and the active action interface, which are trained end-to-end.

3.2 Vision-Language Backbone

We adopt Qwen3.5-4B [34] as our vision-language backbone. It is an early-fusion multimodal model that jointly encodes multi-view images, language instructions, and (in later stages) auxiliary VQA data within a single transformer sequence. The backbone remains fully trainable throughout our recipe.

For pre-training, we additionally construct **Qwen3.5-4B-Action**, a variant we obtain by extending the Qwen3.5-4B tokenizer with a dedicated block of FAST action tokens (see Section 3.3.1). The vocabulary extension is the sole architectural change; the underlying transformer weights are inherited unchanged from Qwen3.5-4B, and the new token embeddings are initialized to the mean of the original embedding table. This lets us reuse Qwen3.5-4B’s pretrained multimodal representations while turning action prediction into standard next-token prediction.

On the visual side, each camera is processed by the same Qwen3.5 ViT under its native smart-resize policy, which preserves aspect ratio and produces on the order of a few hundred visual tokens per view. Pre-training and downstream fine-tuning share this aspect-preserving resolution; a square-cropped alternative is used only for legacy comparison.

3.3 Action Head

We instantiate the action head in two forms, used respectively in the two stages of our training recipe (Section 4): a discrete FAST [31] tokenizer during large-scale pre-training, and a continuous MMDiT flow-matching head with JiT [24] parameterization during mid-training and all downstream fine-tuning. This split reflects two different design pressures: pre-training over hundreds of GPUs and heterogeneous data mixtures rewards an objective that behaves like ordinary next-token prediction, while downstream control rewards a continuous, high-frequency generator that avoids tokenization loss.

3.3.1 FAST: discrete action tokens

FAST [31] encodes an action chunk $a_{t:t+H-1}$ as a short sequence of discrete tokens by first taking a discrete cosine transform along the time axis and then compressing the resulting coefficients with byte-pair encoding. These tokens live inside the Qwen3.5-4B-Action vocabulary (Section 3.2), so action generation is standard next-token prediction on the multimodal prompt, and inference produces an action chunk by greedy decoding followed by FAST detokenization.

We use FAST as the pre-training head for two reasons. First, because the action objective coincides with the VLM’s own pre-training loss, VLA and VLM co-training reduces to weighted next-token prediction on a shared token stream, with no separate loss balancing across regression and language heads and no additional numerical dynamics from a diffusion sampler. Second, the head requires no separate neural decoder beyond the added vocabulary rows, so warm-starting a subsequent stage from a FAST-trained checkpoint is a clean weight transfer of the backbone alone—exactly the operation used when we later swap FAST for MMDiT. The trade-off is that FAST’s tokenization is lossy: continuous action targets are inevitably reconstructed with quantization error, which motivates the continuous head in the next section for the downstream stages that actually control the robot.

3.3.2 MMDiT with JiT parameterization

For mid-training and all downstream fine-tuning, we replace the FAST head with a continuous flow-matching head based on the MMDiT block [11], in the joint-attention configuration used for VLA policies by π_0 [6]. The head jointly processes two token streams: an *observation* stream containing linearly projected VLM hidden states together with a projected proprioceptive-state token, and an *action* stream containing the noisy action chunk a^T . The two streams

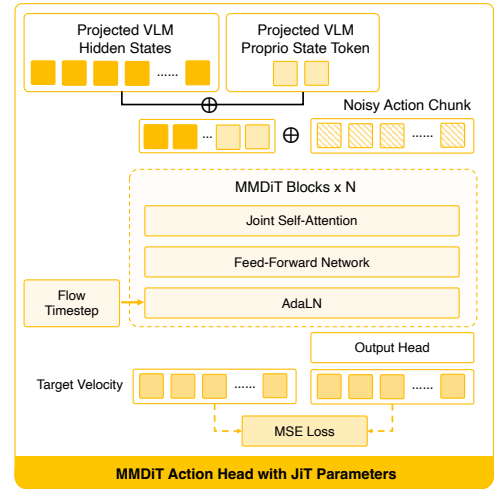


Figure 5: Overview of MMDiT with Jit Parameterization. Overview of MMDiT with Jit Parameterization. Overview of MMDiT with Jit

interact through joint self-attention in N MMDiT blocks. The flow timestep τ modulates the block normalizations through AdaLN. We place the state token on the observation stream (the *condition-side* placement, following π_0); the alternative places it on the action stream. The head architecture is illustrated in Figure 5.

Given a clean action chunk a_0 , Gaussian noise ϵ , and a flow timestep τ , we form the interpolant $a^\tau = \tau a_0 + (1 - \tau)\epsilon$. Standard flow matching [25] trains a network to predict the velocity $v = a_0 - \epsilon$ directly. Following the clean-sample parameterization introduced in JiT [24], our network instead outputs a clean-sample estimate $\hat{a}_0$. At training time, both the prediction and the target are converted to velocity-like quantities through a clamped denominator, and the MSE is computed on those quantities:

$$d_\tau = \max(1 - \tau, t_\epsilon), \quad \hat{v} = \frac{\hat{a}_0 - a^\tau}{d_\tau}, \quad v^* = \frac{a_0 - a^\tau}{d_\tau}, \quad (5)$$

$$\mathcal{L}_{\text{JiT}} = \mathbb{E}_{\tau \sim p(\tau)} \|\hat{v} - v^*\|^2, \quad t_\epsilon = 0.05, \quad (6)$$

where $p(\tau)$ is the same Beta-derived timestep distribution used in our standard flow-matching runs. Applying the same d_τ to both prediction and target means that a perfect $\hat{a}_0 = a_0$ yields exactly zero loss even when the clamp is active. Because $\hat{v} - v^* = (\hat{a}_0 - a_0)/d_\tau$, the objective is equivalently a time-weighted clean-action regression $\|\hat{a}_0 - a_0\|^2/d_\tau^2$: it retains the clean-sample output parameterization while emphasizing low-noise timesteps up to the amplification cap set by t_ϵ .

We adopt this parameterization in all mid-training and downstream fine-tuning runs reported in Section ?? . In our downstream experiments, we found it more stable than direct velocity prediction. One possible explanation is on the output side: clean action chunks may lie closer to the structured trajectory manifold that a coherent action sequence occupies than the velocity targets used in standard flow matching, an argument (author?) [24] make for image denoising and that we conjecture applies here as well.

3.4 Multi-Embodiment Action Space

Our training sources use heterogeneous action conventions, including joint positions and end-effector poses, with different numbers and semantics of control dimensions. We standardize these sources into a common tensor interface for multi-embodiment training. This operation aligns tensor shapes; it does not assume that identically indexed channels carry identical physical semantics across embodiments.

During mid-training, action chunks are represented with a fixed width $d_{\text{max}} = 20$, corresponding to a shared bimanual end-effector layout that the data-curation pipeline (Section 2) maps every source into. For each sample, a binary mask $m \in \{0, 1\}^{d_{\text{max}}}$ identifies the action dimensions present in the source embodiment and is broadcast along the action horizon. Sources with fewer than d_{max} dimensions populate the leading channels and pad the remainder; the masked training objective is

$$\mathcal{L}_{\text{masked}} = \frac{\sum_{h=1}^H \sum_{j=1}^{d_{\text{max}}} m_j \ell_{h,j}}{H \sum_{j=1}^{d_{\text{max}}} m_j}, \quad (7)$$

where $\ell_{h,j}$ is the element-wise JiT loss from Equation 6. Padded dimensions therefore contribute no gradient.

This representation lets one MMDiT trunk and one d_{max} -dimensional projection interface be shared across the mid-training mixture. For downstream adaptation to an embodiment with a different action width, we instantiate embodiment-specific input and output projections, load all shape-compatible backbone and MMDiT parameters, and re-initialize the dimension-dependent projection layers.

3.5 Language Conditioning

Robot episodes in our mid-training corpus, stored in the LeRobot v2.1 format, may provide two levels of language supervision: a *current instruction* describing the local subtask associated with the current action chunk, and an episode-level *task description* describing the overall goal. Our downstream evaluation setup provides task-level instructions, whereas locally aligned annotations are often more fine-grained.

For samples in which both annotations are available, we select the current instruction or task description with equal probability. We refer to this 0.5:0.5 mid-training sampling strategy as *langmix*. Samples with only one available annotation use that annotation directly. Langmix reduces the difference between the granularity of training annotations and the task-level prompts encountered downstream, without changing the underlying action data.

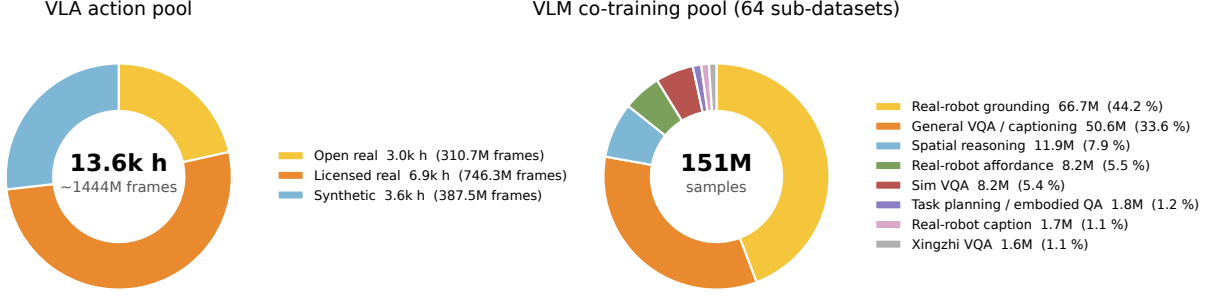


Figure 6: Composition of the pre-training corpus. *Left*: the VLA action pool combines open and externally sourced real, licensed real, and synthetic bimanual manipulation data under a shared LeRobot v2.1 schema. *Right*: the VLM co-training pool aggregates 64 sub-datasets covering grounding, general and spatial VQA, sim-domain VQA, task planning, and affordance supervision. Need to be modified

4 Training and Deployment

Meituan-Robotics-0 is trained in three stages. *Pre-training* (Section 4.1) co-trains the vision-language backbone with a discrete FAST action head on a large heterogeneous corpus of bimanual manipulation data, allowing action learning to reuse the backbone’s next-token objective. *Mid-training* (Section 4.2) replaces FAST with the continuous MMDiT flow-matching head introduced in Section 3.3.2, and continues co-training to adapt the backbone to this action interface. Post-training then adapts the stage-2 checkpoint to downstream tasks before deployment.

4.1 Pre-training

4.1.1 Recipe

Pre-training is intended to produce a *backbone*, not a deployable policy. Every optimization step consumes one VLA batch and one VLM batch, combined as

$$\mathcal{L} = \lambda_{\text{vla}} \mathcal{L}_{\text{action}} + \lambda_{\text{vlm}} \mathcal{L}_{\text{vqa}}, \quad \lambda_{\text{vla}}=1.0, \quad \lambda_{\text{vlm}}=0.1, \quad (8)$$

where $\mathcal{L}_{\text{action}}$ is the FAST next-token cross-entropy over the extended Qwen3.5-4B-Action vocabulary (Section 3.3.1) and $\mathcal{L}_{\text{vqa}}$ is the standard next-token loss on the co-training VQA pool. We include the VQA objective to retain general vision-language supervision during action pre-training. The pre-training schedule spans 500k optimization steps on 256 GPUs and uses AdamW with cosine learning-rate decay.

4.1.2 Data

The VLA action pool combines three source groups in a single LeRobot v2.1 schema [7] with three camera streams and a 20-dimensional bimanual end-effector action space, contributing ≈ 13.6 k hours ($\approx 1,444$ M frames) of manipulation data in total.

Open and externally sourced real data draws from AgiBot World [1], Galaxea Open-World [16], RoboCOIN [43], LET, and RoboMIND [42], contributing 310.7 M frames across a broad set of arm and gripper morphologies.

Licensed real data adds seven proprietary embodiments and a further 746.3 M frames, making it the largest contributor to the mixture.

Synthetic data covers rollouts from RoboTwin 2.0 [29], Genie Sim 3.0 [49], and two InternData source releases, adding 387.5 M frames of bimanual trajectories.

State and action channels are min–max normalized using embodiment-specific statistics, and gripper values are binarized. For language conditioning, we use the first available instruction annotation for each sample.

The VLM co-training pool aggregates 64 sub-datasets (≈ 150.7 M samples) spanning real-robot grounding and captioning, general and spatial visual question answering, sim-domain VQA, task planning, and affordance supervision. Figure 6 summarizes both pools.

4.2 Mid-training

Mid-training reuses the pre-training co-training scheme but swaps the FAST head for the continuous MMDiT head with JiT parameterization introduced in Section 3.3.2. We transfer the backbone weights from the pre-training checkpoint and initialize the MMDiT head from scratch, since the discrete and continuous action interfaces do not share a decoder. Mid-training uses 128 GPUs and retains the AdamW optimizer and cosine schedule, with a lower learning rate for the pre-trained backbone and a higher learning rate for the newly initialized action head.

Mid-training uses the *langmix* scheme introduced in Section 3.5, exposing the policy to both fine-grained current instructions and task-level descriptions.

The mid-training mixture removes one large synthetic sub-corpus (≈ 340 M frames), subsamples each of the two largest general-VQA sources to 10%, and weights VLA sources by the square root of their frame counts. These adjustments reduce the relative dominance of the largest sources while retaining the remaining real, synthetic, and VQA data from pre-training.

4.3 Real-World Post-Training

Starting from the mid-training checkpoint, real-robot post-training proceeds in three stages: supervised fine-tuning (SFT) by behavioral cloning (BC) on successful teleoperated demonstrations, HG-Dagger-style data aggregation with human-in-the-loop (HIL) coaching, and offline reinforcement learning (RL) using RECAP. All three stages share the same observation and action representation and the same data preparation procedure, described in Section A.

Mid-training provides general manipulation priors, but reliable closed-loop execution requires post-training, and each stage contributes a distinct kind of real-robot data. Teleoperated demonstrations show the nominal behavior, yet small action errors accumulate along a trajectory and push the robot outside the demonstration distribution, where the policy has never been supervised. Human corrections supply exactly this missing supervision, coaching the policy out of the failure states. Rollouts themselves require no operator effort, but they mix progress-making with harmful behavior, and cloning them would only reproduce the current policy. RL makes them useful by assigning credit on individual actions, so the policy distinguishes quality of its own behaviors.

4.3.1 Supervised Fine-Tuning

Starting from the mid-training checkpoint π_{mid} , we train an initial task policy π_0 on $\mathcal{D}_{\text{demo}}$, the set of successful teleoperated demonstrations, using BC with the supervised action-prediction objective. This policy acquires the basic task behavior but is not yet reliable in autonomous execution: common failures include inaccurate grasping, placement offsets, halting or repeated actions, and error accumulation over long task sequences.

4.3.2 HG-Dagger-Style Human Intervention

The initial SFT policy is trained only on successful executions. During autonomous operation, small prediction errors can move the robot into states that are rarely seen in those demonstrations, leaving the policy with little supervision for recovery. We address this distribution shift using the human-gated data-aggregation component of HG-Dagger [18]: the current policy induces states through real-robot rollouts, a human takes control when correction is needed, and the resulting recovery trajectories are aggregated for policy retraining. The original HG-Dagger method additionally learns an ensemble-uncertainty threshold from intervention events; we do not use that component and therefore refer to our procedure as HG-Dagger-style.

Human-gated data collection. We deploy the current MR0 policy while a human operator monitors its execution. Let $g_t \in \{0, 1\}$ denote the operator-controlled gate, where $g_t = 0$ assigns control to the current policy and $g_t = 1$ assigns control to the human policy π_H . The action executed at time t is

$$a_t^{\text{exec}} = (1 - g_t) \pi_{k-1}(o_t) + g_t \pi_H(o_t). \quad (9)$$

Initially, our policy roll outs automatically. If the policy halts, repeats ineffective actions, or produces a clearly incorrect motion, the human operator takes control over the recovery segment and manually returns control after reaching a safe state that autonomous execution can safely continue. A single episode may contain multiple interventions. Episodes with failures that cannot be recovered through human control are excluded from the HG-Dagger training set, but they are retained with a failure outcome for RECAP value learning (section 4.3.3).

Algorithm 1 HG-DAGger-style real-robot post-training in MR0**Require:** Mid-training checkpoint π_{mid} , demonstration dataset $\mathcal{D}_{\text{demo}}$, rounds K

- 1: $\mathcal{D}^{(0)} \leftarrow \mathcal{D}_{\text{demo}}$
- 2: Train π_0 from π_{mid} on $\mathcal{D}^{(0)}$
- 3: **for** $k = 1$ to K **do**
- 4: Deploy π_{k-1} under operator gating (eq. (9)); collect interventions $\mathcal{D}_{\text{HIL}}^{(k)}$
- 5: $\mathcal{D}^{(k)} \leftarrow \mathcal{D}^{(k-1)} \cup \mathcal{D}_{\text{HIL}}^{(k)}$
- 6: Train π_k from π_{mid} on $\mathcal{D}^{(k)}$
- 7: **end for**

Ensure: Final task policy π_K

From the complete rollout, the HG-DAGger training set keeps only the samples collected while $g_t = 1$, whereas RECAP keeps both control sources and uses the advantage label to route the prompt rather than to select samples. For round k , the added data for HG-DAGger includes only the HIL data as supervision:

$$\mathcal{D}_{\text{HIL}}^{(k)} = \{(o_t, a_t^H) \mid g_t = 1\}. \quad (10)$$

Iterative retraining with cumulative HIL data. We repeat rollout, intervention, and retraining for K rounds. Let $\mathcal{D}_{\text{HIL}}^{(k)}$ denote the intervention data collected by deploying π_{k-1} . Retaining HIL data from all previous rounds, the training set for round k is

$$\mathcal{D}^{(k)} = \mathcal{D}_{\text{demo}} \cup \bigcup_{i=1}^k \mathcal{D}_{\text{HIL}}^{(i)}. \quad (11)$$

We train π_k on $\mathcal{D}^{(k)}$ by SFT with the same action-prediction objective used for π_0 , deploy it to collect $\mathcal{D}_{\text{HIL}}^{(k+1)}$, and repeat. Every round is trained from π_{mid} , not from the previous round’s fine-tuned policy. Algorithm 1 summarizes the loop.

Section 6.2.1 gives the number of rounds and trajectories used for each evaluated task.

The two sources supply complementary supervision: demonstrations contain complete, successful executions and preserve the full task procedure, whereas HIL data contain human corrections to policy errors and accumulate examples of difficult states and recovery behavior.

4.3.3 RECAP: Advantage-Conditioned Policy Learning

RECAP (RL with Experience and Corrections via Advantage-conditioned Policies) is an offline reinforcement-learning post-training framework for real-world robot policies [2]. We apply RECAP after SFT and human-gated data aggregation to learn jointly from demonstrations, autonomous rollouts, task outcomes, and human corrections. The resulting dataset contains both near-optimal and suboptimal behavior, and autonomous improvement requires learning not only which actions led to success, but also which actions caused delays or failures. Unlike the preceding BC updates, which directly clone selected human actions, RECAP uses outcome-derived value and advantage estimates to distinguish the relative quality of action chunks throughout the aggregated dataset.

We adopt advantage conditioning rather than likelihood-based policy-gradient RL for the reasons given by (author?) [2]: it reuses heterogeneous off-policy data instead of requiring fresh on-policy rollouts, the scarcest resource on real hardware, and it does not require an action log-likelihood, which PPO [36]- and SAC [14]-style updates need but flow-matching policies cannot evaluate. In the policy update, the RL signal enters only through an advantage-derived conditioning token, while every sample retains the standard supervised action-prediction loss.

Iterative aggregation. RECAP resumes from the final HG-DAGger policy π_K and its dataset $\mathcal{D}^{(K)}$ (eq. (11)). At iteration m we deploy the current policy autonomously, record each episode together with its success or failure outcome as $\mathcal{D}_{\text{auto}}^{(m)}$, and aggregate

Algorithm 2 RECAP post-training in MR0

Require: Mid-training checkpoint π_{mid} , value initialization V_{init} , HG-Dagger policy π_K and dataset $\mathcal{D}^{(K)}$, iterations M

- 1: $\pi_0^{\text{RL}} \leftarrow \pi_K, \quad \mathcal{D}_{\text{RL}}^{(0)} \leftarrow \mathcal{D}^{(K)}$
- 2: **for** $m = 1$ to M **do**
- 3: Deploy π_{m-1}^{RL} autonomously, under guidance scale β (eq. (18)) once $m > 1$; collect rollouts and outcomes $\mathcal{D}_{\text{auto}}^{(m)}$
- 4: $\mathcal{D}_{\text{RL}}^{(m)} \leftarrow \mathcal{D}_{\text{RL}}^{(m-1)} \cup \mathcal{D}_{\text{auto}}^{(m)}$ (eq. (12))
- 5: Fit V_m from V_{init} on $\mathcal{D}_{\text{RL}}^{(m)}$ using outcome-derived targets (eq. (37))
- 6: Label every valid start with $I_t = \mathbb{I}[A_t^{(N)} \geq \epsilon]$ under V_m (eqs. (14) and (15))
- 7: Train π_m^{RL} from π_{mid} on $\mathcal{D}_{\text{RL}}^{(m)}$, conditioning positive starts on c_t with dropout p_{drop}
- 8: **end for**

Ensure: Final task policy π_M^{RL} , deployed under guidance scale β

$$\mathcal{D}_{\text{RL}}^{(m)} = \mathcal{D}^{(K)} \cup \bigcup_{i=1}^m \mathcal{D}_{\text{auto}}^{(i)}. \quad (12)$$

The aggregation mirrors eq. (11), but the increment differs: eq. (10) keeps only human-controlled frames, whereas $\mathcal{D}_{\text{auto}}^{(m)}$ retains complete episodes, failures included. The terminal outcome is what supervises the value function, and low-advantage segments still train the unconditional branch.

Algorithm 2 summarizes our instantiation, adapted from [2]; the following paragraphs define its value targets, advantage labels, conditional policy training, and guided inference. As in Algorithm 1, each iteration deploys the policy from the preceding iteration, appends what it collects, and retrains from a fixed initial checkpoint rather than continuing from the preceding iteration’s weights. We write π_m^{RL} for the RECAP policies to distinguish them from the SFT and HG-Dagger policies $\pi_0, \dots, \pi_K$.

Value-function learning. The value function $V^{\pi_{\text{ref}}}(o_t, \ell)$ estimates the expected return from observation o_t under instruction ℓ , where the reference policy π_{ref} is the behavior distribution of the aggregated dataset: demonstrations, human corrections, and autonomous trajectories from previously deployed policies. Episode outcomes are converted into per-timestep rewards and normalized to scalar targets $y_t \in [-1, 0]$, with 0 at successful completion and a terminal penalty separating failures. Following distributional value learning [3], the network predicts a categorical distribution over $B = 201$ atoms rather than regressing y_t directly, and $V(o_t, \ell)$ is its mean. Section B gives the reward, the normalization, and the projection onto the atom support.

N -step advantage estimation. Because the value predictions are normalized to $[-1, 0]$, rewards for advantage estimation are expressed on the same scale and derived from adjacent scalar targets:

$$\tilde{r}_t = \begin{cases} y_t - y_{t+1}, & t < T, \\ y_T, & t = T. \end{cases} \quad (13)$$

For a complete N -step horizon, the advantage is

$$A_t^{(N)} = \sum_{k=0}^{N-1} \tilde{r}_{t+k} + V(o_{t+N}, \ell) - V(o_t, \ell). \quad (14)$$

If fewer than N steps remain, rewards are accumulated through the terminal state and the bootstrap value beyond termination is set to zero. A higher advantage indicates that the segment improves task prospects relative to the value expected at its start. We binarize it as

$$I_t = \mathbb{I}[A_t^{(N)} \geq \epsilon]. \quad (15)$$

In our experiments, $N = 50$ matches the action horizon $H = 50$. Consequently, I_t evaluates the complete action chunk $\mathbf{a}_{t:t+H}$ beginning at frame t . Labels at frames $t + 1, \dots, t + 49$ do not alter that sample; they apply when those frames are themselves valid starts. We set ϵ to the 70th percentile over the aggregated dataset, yielding approximately 30% positive-labeled starts.

Positive advantage-conditioned policy learning. Crucially, I_t is a chunk-level prompt-routing label attached to the valid start t , not a loss mask. For either value of I_t , the supervised target remains the complete 50-step chunk $\mathbf{a}_{t:t+H}$; the indicator only determines whether that sample is eligible to receive the positive condition.

The binary indicator determines the advantage condition

$$c_t = \begin{cases} \text{Advantage: positive,} & I_t = 1, \\ \emptyset, & I_t = 0, \end{cases} \quad (16)$$

which is appended to the task instruction after a newline. During training, the positive condition is independently dropped for each positive sample with probability $p_{\text{drop}} = 0.3$. Thus, a positive-labeled start uses the explicit condition with probability 0.7 and the base prompt with probability 0.3, whereas a start with $I_t = 0$ always uses the base prompt. Each sample traverses exactly one route in a forward pass and is not duplicated across branches.

All chunks, regardless of I_t , contribute to the same supervised action-prediction loss. Unlike approaches that discard low-quality samples or weight the action loss directly by advantage, RECAP retains every selected training example and lets the advantage change only the textual conditioning input. Low-advantage behavior is therefore not removed: it teaches the unconditional branch, together with the positive examples whose conditions are dropped, so that branch models the complete behavior distribution while the conditioned branch captures the direction toward higher-advantage behavior. The method does not train an explicit negative conditional direction.

Human-intervention frames are assigned $I_t = 1$ because the corrective actions are treated as expert behavior. For RECAP policy training, demonstration frames are also labeled positive; the same condition-dropout rule still supplies unconditional examples. As in the HG-Dagger rounds, every iteration is fine-tuned from a fixed initial checkpoint— π_{mid} for the policy and V_{init} for the value function—rather than continued from the models produced in the preceding iteration, so only the dataset carries over and sequential drift is reduced.

Classifier-free-guided inference. Condition dropout lets one model represent both the positive-conditioned policy $\pi_{\theta}(a \mid I = 1, o, \ell)$ and the unconditional policy $\pi_{\theta}(a \mid o, \ell)$. Their ratio defines the guided target policy [2, 12]

$$\hat{\pi}(a \mid o, \ell) \propto \pi_{\theta}(a \mid o, \ell) \left(\frac{\pi_{\theta}(a \mid I = 1, o, \ell)}{\pi_{\theta}(a \mid o, \ell)} \right)^{\beta}, \quad (17)$$

where β is the guidance scale: $\beta = 1$ recovers the positive-conditioned branch, and $\beta > 1$ extrapolates beyond it. Equation (17) is a statement about densities, which a flow-matching policy cannot evaluate. We therefore approximate it in action space: a complete chunk is denoised in parallel through the base and positive-conditioned branches, and the two resulting chunks are extrapolated once, as

$$\mathbf{a}_{\text{CFG}} = \mathbf{a}_{\text{base}} + \beta (\mathbf{a}_{\text{pos}} - \mathbf{a}_{\text{base}}), \quad \beta = 2.5. \quad (18)$$

Here $\mathbf{a}_{\text{base}} \sim \pi_{\theta}(\cdot \mid o_t, \ell)$ and $\mathbf{a}_{\text{pos}} \sim \pi_{\theta}(\cdot \mid I = 1, o_t, \ell)$. Both branches start from the same initial noise, so their difference is driven by the advantage condition rather than by independent sampling noise. Because eq. (18) extrapolates the two completed chunks rather than combining their velocity fields during denoising, it is a one-shot approximation: $\mathbf{a}_{\text{CFG}}$ is the executed chunk and is not in general a sample from $\hat{\pi}$. Deployment therefore steers the prediction toward the learned positive direction even though low-advantage behavior remains part of unconditional training.

4.4 Deployment

4.4.1 Action Chunk Merging

Overview and motivation. MR0 predicts a chunk of future dual-arm end-effector (EEF) actions at each inference call. Model inference and robot control are executed asynchronously: the inference thread periodically acquires the latest observation and generates a new action chunk, while the control thread continuously dispatches actions from the current execution buffer at 30 Hz.

Each predicted EEF chunk is converted into a joint-space trajectory using continuous inverse kinematics (IK). Within a chunk, the solution from the preceding timestep is used to initialize the IK solver at the next timestep, improving local joint-space continuity. The resulting dual-arm action is represented as

$$\mathbf{q}_i = [\mathbf{q}_{i,1:6}^L, g_i^L, \mathbf{q}_{i,1:6}^R, g_i^R] \in \mathbb{R}^{14}, \quad (19)$$

where $\mathbf{q}_{i,1:6}^L$ and $\mathbf{q}_{i,1:6}^R$ denote the arm-joint positions, while g_i^L and g_i^R denote the continuous left and right gripper positions.

Successive action chunks are generally not numerically identical because each prediction is conditioned on an updated observation. Changes in the robot state, visual observation, object state, and tracking error may cause a newly predicted chunk to encode a closed-loop correction relative to the residual trajectory in the current execution buffer. Directly replacing the current buffer would apply this correction within a single 33.3 ms control period, potentially introducing large joint velocity, acceleration, and jerk at the chunk boundary.

To mitigate discontinuities at chunk boundaries, we first remove the stale prefix of each newly predicted chunk and then linearly blend the temporally overlapping portions of the old and new trajectories [50, 45].

Runtime temporal alignment and prefix trimming. Let $\mathbf{Q}_o$ denote the old action sequence supplied to the blending stage, derived from the unexecuted portion of the current action buffer, and let $\mathbf{Q}_n$ denote the newly predicted joint-space chunk. The client maintains a runtime consumption counter k . Each successful call to `pop_next_action()` increments k by one. Therefore, when a new chunk arrives, k records the number of actions successfully removed from the execution buffer for dispatch since the previous successful merge.

Unlike a fixed delay estimate derived from the nominal inference frequency, k is measured online for each merge. It therefore reflects variations in inference latency, thread scheduling, and control-loop timing. Note that k measures command consumption on the control side rather than completion verified from physical robot feedback.

The stale-prefix length d , trimmed new chunk $\hat{\mathbf{Q}}_n$, and overlap length L are defined jointly as

$$d = \min(k, K), \quad \hat{\mathbf{Q}}_n = \mathbf{Q}_n[d:], \quad L = \min(|\mathbf{Q}_o|, |\hat{\mathbf{Q}}_n|), \quad (20)$$

where K is a configurable upper bound on prefix trimming. The effective trim length is determined by the measured counter k , while K limits the maximum number of actions removed from the beginning of the new chunk.

The trimming cap prevents an unusually large consumption count from automatically removing the same number of actions from the new prediction. Such a large k may occur after an inference stall, communication delay, or temporary scheduling disruption. Without a cap, setting $d = k$ could leave too few future actions for subsequent execution or discard the new chunk entirely. When K is selected below the prediction horizon, it limits this loss of future actions and reduces the risk of exhausting the execution buffer.

This protection introduces a trade-off. When $k \leq K$, the stale prefix is removed according to the measured command-consumption progress. When $k > K$, only the first K actions are removed. This preserves a longer suffix of the new prediction, but the remaining $k - K$ consumed steps are not compensated for by prefix trimming. Thus, K is an engineering guard against excessive trimming rather than a guarantee of complete temporal alignment.

If $d \geq |\mathbf{Q}_n|$, the new prediction is regarded as fully stale and is ignored. The existing execution buffer and the current consumption counter are retained in this case.

Linear overlap blending. Let $\mathbf{q}_i^o$ and $\hat{\mathbf{q}}_i^n$ denote the i -th action in the residual old trajectory and the i -th action in the trimmed new trajectory, respectively. For each overlap index $i = 0, \dots, L - 1$, the two actions are combined as

$$w_i = 1 - \frac{i}{\max(L - 1, 1)}, \quad \tilde{\mathbf{q}}_i = w_i \mathbf{q}_i^o + (1 - w_i) \hat{\mathbf{q}}_i^n. \quad (21)$$

For $L > 1$, the coefficient w_i decreases linearly from one to zero. The first action in the overlap is therefore inherited entirely from the residual old trajectory, whereas the final overlapping action is inherited entirely from the trimmed new trajectory. For $L = 1$, the single overlapping action is inherited from the old trajectory, consistent with the implementation.

The blended overlap is concatenated with the non-overlapping suffix $\hat{\mathbf{Q}}_n[L:]$ to form the updated execution buffer. When $L = 0$, the blended sequence is empty and the trimmed new trajectory is used directly. After a successful merge, the runtime consumption counter k is reset to zero and begins counting actions successfully popped for dispatch before the next chunk integration.

Algorithm 3 Asynchronous Action-Chunk Merging with Online Prefix Trimming

Require: Old residual joint-space sequence $\mathbf{Q}_o$, new EEf-space action chunk $\mathbf{a}_n$, runtime consumption counter k , trim cap K , and measured joint state $\mathbf{q}_a$ used as the initial IK seed

Ensure: Updated execution buffer $\mathbf{Q}_b$ and consumption counter k

```

1:  $\mathbf{Q}_n \leftarrow \text{ContinuousIK}(\mathbf{a}_n; \mathbf{q}_a)$ 
2:  $d \leftarrow \min(k, K)$ 
3: if  $d \geq |\mathbf{Q}_n|$  then
4:    $\mathbf{Q}_b \leftarrow \mathbf{Q}_o$ 
5:   return  $\mathbf{Q}_b, k$ 
6: end if
7:  $\hat{\mathbf{Q}}_n \leftarrow \mathbf{Q}_n[d:]$ 
8:  $L \leftarrow \min(|\mathbf{Q}_o|, |\hat{\mathbf{Q}}_n|)$ 
9: if  $L = 0$  then
10:   $\mathbf{Q}_b \leftarrow \hat{\mathbf{Q}}_n$ 
11:   $k \leftarrow 0$ 
12:  return  $\mathbf{Q}_b, k$ 
13: end if
14: for  $i = 0$  to  $L - 1$  do
15:    $w_i \leftarrow 1 - \frac{i}{\max(L - 1, 1)}$ 
16:    $\tilde{\mathbf{q}}_i \leftarrow w_i \mathbf{Q}_o[i] + (1 - w_i) \hat{\mathbf{Q}}_n[i]$ 
17: end for
18:  $\mathbf{Q}_b \leftarrow [\tilde{\mathbf{q}}_0, \dots, \tilde{\mathbf{q}}_{L-1}] \parallel \hat{\mathbf{Q}}_n[L:]$ 
19:  $k \leftarrow 0$ 
20: return  $\mathbf{Q}_b, k$ 

```

For compactness, algorithm 3 presents the core merge path; implementation-specific construction of $\mathbf{Q}_o$ and other boundary handling are omitted for clarity. $\mathbf{a}_n$ denotes the newly predicted EEf chunk, $\mathbf{q}_a$ denotes the measured robot joint state, and $\mathbf{Q}_b$ denotes the updated execution buffer. Continuous IK is performed before buffer integration. The subsequent prefix trimming, blending, and buffer replacement operations are protected by a buffer lock.

Smoothness and inference-rate selection. The linear handover distributes the discrepancy between two chunks over L control steps. For $L > 1$, the mixing coefficient changes by $1/(L - 1)$ per step. A larger L therefore introduces the new prediction more gradually, provided that the old and new trajectories are themselves locally smooth. However, it also preserves the influence of the old trajectory for a longer duration. The overlap length thus introduces a trade-off between smoothness and responsiveness to the latest observation.

The inference rate must be selected jointly with the 30 Hz control rate. If the inference frequency is too high, only a small number of actions are dispatched before the next merge, and the linear transition may restart before the latest prediction receives substantial execution weight. Conversely, if the inference frequency is too low, updated observations and closed-loop corrections enter the control loop more slowly.

In practice, we use an inference frequency in the range of 2–3 Hz. Together with the 30 Hz control frequency, this corresponds nominally to approximately 10–15 control steps between consecutive chunk integrations. This nominal range is not used as a fixed trimming offset. The actual number of actions popped from the execution buffer is measured online through k and may vary across inference calls.

This operating range provides a practical compromise among observation refresh, trajectory continuity, and the rate at which new predictions enter the execution buffer. It depends on the control frequency, action horizon, overlap length, trimming cap, and inference latency rather than being an intrinsic property of MR0.

Before real-robot execution, three warm-up chunks are generated to initialize the model and communication pipeline. Since these chunks are not executed, they are discarded after warm-up. The action buffer is then anchored to the measured robot joint state, ensuring that the first online chunk is blended from the physical robot configuration rather than from an unexecuted predicted trajectory.

Linear overlap smoothing reduces abrupt changes in discrete joint-position commands but does not explicitly guarantee velocity continuity, acceleration continuity, collision avoidance, or dynamic feasibility. It is therefore used as an execution-side trajectory-handover mechanism rather than as a complete motion planner.

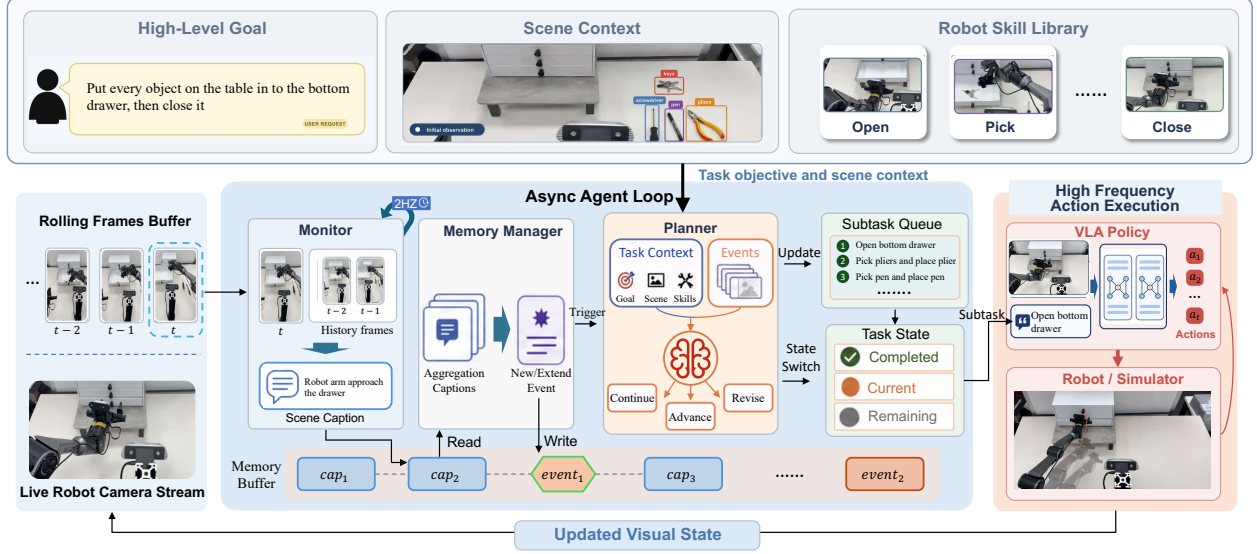


Figure 7: Overview of the event-driven embodied agent. The Monitor converts visual observations into captions, the Memory Manager aggregates them into task-relevant events, and triggered events invoke the Planner to update the subtask executed by the VLA policy.

5 Embodied Agent System

We develop an Event-driven Hierarchical Embodied Agent System that performs task decomposition, progress assessment, and closed-loop planning on top of a low-level vision-language-action policy. The system organizes high-level reasoning around three key components: (1) a Monitor that converts short visual windows into structured captions to track low-level execution; and (2) a Memory Manager that aggregates captions into task-relevant events and determines when Planner reassessment is needed; (3) a Planner that decomposes the user goal into executable subtasks and reassesses the task state when necessary. The active subtask conditions the low-level policy for continuous action generation, while new observations are fed back through the Monitor and Memory Manager to close the perception-action loop.

5.1 Event-Driven Task Planning and Execution

Let g denote the user goal, s_0 the initial scene context, and $\mathcal{S}$ the available robot skill library. Given $(g, s_0, \mathcal{S})$, the Planner first generates an initial executable sequence of subtasks,

$$\mathcal{Q}^{\text{init}} = (\bar{q}_1, \bar{q}_2, \dots, \bar{q}_N), \quad (22)$$

where $\bar{q}_i$ denotes the i -th subtask in the initial plan. During execution, the agent maintains a dynamic task state $\mathcal{X}_t = (q_t, R_t)$, where q_t denotes the active subtask at time t and R_t denotes the ordered sequence of remaining subtasks. The active subtask q_t is provided to the low-level VLA policy as its task instruction. Given the current visual observation o_t , the policy π_{VLA} generates the corresponding robot actions

$$a_t = \pi_{\text{VLA}}(o_t, q_t). \quad (23)$$

Rather than invoking the Planner at every observation step, the agent performs high-level reasoning only when task-relevant events arise during execution. The Monitor summarizes a short temporal window of visual observations W_t in the context of the active subtask q_t , while the Memory Manager consolidates these summaries into task-relevant events. An event E_t that signals meaningful progress, execution failure, or a relevant change in the environment triggers the Planner to reassess and update the task state $\mathcal{X}_t$. If no such event is detected, the current subtask remains active and low-level execution continues. This event-driven design decouples continuous visual monitoring and action execution from sparse high-level reasoning.

5.2 Monitor

The Monitor tracks the execution of the low-level VLA policy by converting short temporal windows of visual observations into structured scene captions. Given the active subtask q_t and the visual observation window W_t at time t ,

the Monitor produces

$$C_t = \mathcal{M}(W_t, q_t), \quad (24)$$

where C_t describes the observed execution progress and task-relevant changes within the current visual window. The active subtask provides task context that directs the Monitor toward relevant objects, locations, and interactions.

The Monitor reports visually grounded observations without determining whether the active subtask has been completed or whether the task state should be changed. These decisions are deferred to the subsequent event-level reasoning process. Using a short temporal window rather than an isolated frame allows the Monitor to capture local state transitions during VLA execution. The resulting captions form a temporally ordered visual trace that is subsequently processed by the Memory Manager.

5.3 Memory Manager

The Memory Manager serves as an intermediate semantic layer between continuous visual perception and high-level task reasoning. Rather than forwarding individual captions directly to the Planner, it aggregates the fine-grained caption stream into task-relevant events that capture execution progress and meaningful changes in the task state.

The captions produced by the Monitor are stored in a temporally ordered caption buffer. At each Memory Manager update, $\mathcal{B}_t$ denotes the captions that are currently available but have not yet been processed. The Memory Manager maintains an ordered event memory $\mathcal{H}$. Prior to the update at time t , the available event history is denoted by

$$\mathcal{H}_{<t} = [(E_{\tau_1}, K_{\tau_1}), \dots, (E_{\tau_m}, K_{\tau_m})], \quad \tau_i < t, \quad (25)$$

where each entry consists of an event representation E_{τ_i} and its associated visual evidence K_{τ_i} .

Given the newly available caption batch $\mathcal{B}_t$, the preceding event history $\mathcal{H}_{<t}$, the current task state $\mathcal{X}_t$, and the user goal g , the Memory Manager produces an event representation E_t and predicts a binary trigger z_t indicating whether Planner reassessment is required,

$$(E_t, z_t) = \mathcal{G}(\mathcal{B}_t, \mathcal{H}_{<t}, \mathcal{X}_t, g), \quad z_t \in \{0, 1\}. \quad (26)$$

The resulting event either updates the most recent entry in $\mathcal{H}$ when the new observations extend the ongoing interaction, or is appended as a new entry when they indicate a distinct task-relevant change. The associated visual evidence K_t is retained together with E_t to preserve the visual basis of the event representation.

Event-memory update and Planner invocation are decoupled. The trigger variable z_t determines whether the resulting event warrants high-level reassessment. Routine execution progress can therefore be incorporated into $\mathcal{H}$ without invoking the Planner, while events indicating subtask completion, execution failure, changes in task preconditions, or other plan-relevant state changes trigger Planner reassessment.

5.4 Planner

The Planner supports two complementary forms of high-level reasoning. Task planning generates an executable sequence of subtasks from the current task state and scene context under the user goal and available robot skills. We denote a plan generated at time t by $Q^{(t)}$. At task initialization, the Planner generates the initial plan as

$$Q^{\text{init}} = Q^{(0)} = \mathcal{P}^{\text{plan}}(g, s_0, \mathcal{S}). \quad (27)$$

During execution, the same planning function is invoked when the current plan requires revision. The updated scene context and task state are then used to generate a revised subtask sequence $Q^{(t)}$.

During closed-loop execution, the Planner is selectively invoked for event-conditioned decision making. When $z_t = 1$, it receives the event E_t , its associated visual evidence K_t , the current task state $\mathcal{X}_t$, and the user goal g , and produces a task-level decision

$$d_t = \mathcal{P}^{\text{decide}}(E_t, K_t, \mathcal{X}_t, g). \quad (28)$$

The decision determines how execution should proceed under the observed task state. It may preserve the active subtask, advance to the next subtask when sufficient progress has been established, or determine that the remaining plan is no longer appropriate. In the latter case, $\mathcal{P}^{\text{plan}}$ is invoked to generate a revised plan from the updated task and scene state.

The Planner assesses task progress from the observed state rather than requiring the complete action trajectory to be observed. The associated keyframes K_t provide visual evidence for validating the event representation E_t . The Planner adopts a conservative decision policy and preserves the current execution state when the available evidence is insufficient to support a task-level transition.

Table 3: Success rate (%) on RoboTwin 2.0 (50 tasks, 100 episodes/task per split). Baseline numbers are compiled from (author?) [32, 33, 47]; entries marked “—” are not reported in the corresponding source. Overall is the mean of Clean and Randomized. **Bold**: best; underlined: second best.

Method	Clean	Randomized	Overall
π_0 [6]	65.9	58.4	62.2
$\pi_{0.5}$ [5]	82.7	76.8	79.8
GR00T N1.6 [30]	47.6	—	—
X-VLA [44]	72.8	72.8	72.8
ABot-M0 [47]	86.1	85.1	85.6
Qwen-VLA-Instruct [33]	86.1	87.2	86.7
LingBot-VLA [41]	86.5	85.3	85.9
Motus [4]	88.7	87.0	87.8
StarVLA [38]	88.7	87.8	88.3
InternVLA-A1 [9]	89.4	89.6	89.5
Being-H0.7 [27]	90.2	89.6	89.9
Fast-WAM [51]	91.9	91.8	91.8
LingBot-VA [23]	92.9	91.5	92.2
InternVLA-A1.5 [32]	93.3	<u>93.0</u>	<u>93.2</u>
Meituan-Robotics-0 (Ours)	<u>93.1</u>	93.6	93.4

5.5 Asynchronous Execution Pipeline

The Monitor, Memory Manager, and Planner operate asynchronously at different temporal scales. The Monitor processes visual observations at a relatively high frequency, while the Memory Manager updates event representations from newly available captions and the Planner is invoked only when high-level reassessment is required. Multiple Monitor requests can be evaluated concurrently to improve visual inference throughput. Since their completion order may differ from the temporal order of the underlying observations, the resulting captions are committed to the Memory Manager according to observation time, preserving temporal consistency for event construction.

Planner inference does not interrupt visual monitoring or low-level execution. Captions generated while the Planner is active remain buffered and are incorporated into subsequent Memory Manager updates. Task-state updates are applied sequentially after each Planner decision, preventing concurrent high-level decisions from modifying the same execution state. This asynchronous design preserves continuous perception and action execution while allowing high-level reasoning to operate sparsely in response to task-relevant events.

6 Experiments

6.1 Simulation

6.1.1 RoboTwin 2.0

RoboTwin 2.0 [29] is a bimanual desktop-manipulation benchmark of 50 tasks in a 14-D joint-space action interface. Each task provides 50 demonstrations in a clean-scene split (`demo_clean`) and 500 demonstrations in a heavily randomized split (`demo_randomized`) that varies backgrounds, table clutter, table height, and lighting. All 50 tasks are trained jointly. We fine-tune Meituan-Robotics-0 end-to-end from the stage-2 checkpoint using native-resolution inputs, and evaluate at 100 episodes per task on both splits (5000 episodes per split).

Table 3 reports the main comparison. Meituan-Robotics-0 attains 93.4% overall success, on par with the strongest concurrent baseline InternVLA-A1.5 (93.2%) and improving over the next tier—LingBot-VA, Fast-WAM, Being-H0.7—by 1.2 to 3.5 points. Compared with the earlier generation of desktop-bimanual VLA reports (ABot-M0, Qwen-VLA-Instruct, $\pi_{0.5}$), the improvement ranges from 6.7 to 13.6 points overall. Performance is comparable across the Clean and Randomized splits, with a gap of 0.5 percentage points.

Two observations from Table 3 are worth flagging. First, the head architecture we use—MMDiT with condition-side state placement and JiT parameterization (Section 3.3.2)—is close in spirit to what ABot-M0 calls Action Manifold Learning: both predict a clean action sample and evaluate the loss in v -space. The performance gap suggests that this shared parameterization alone does not explain the result. Second, most strong entries in the table are specialist policies fine-tuned per benchmark; our result shows that downstream adaptation from a shared stage-2 representation is also effective in this setting.

Table 4: **RoboDojo simulation results.** Each entry is reported as Score/SR, where Score captures partial task progress and SR denotes the binary task-completion rate (%). Generalization is the equal-weighted average of Gen-Std and Gen-Rand. Methods are grouped by their primary modeling design and use of UMI pre-training. Bold indicates the best result within each group.

Model	Avg. $\uparrow$	Generalization $\uparrow$	Precision $\uparrow$	Long-Horizon $\uparrow$	Memory $\uparrow$	Open $\uparrow$
<i>Reasoning-augmented</i>						
GalaxeaVLA (G0) [17]	5.82/2.96	4.54/3.00	8.10/3.83	12.60/5.58	3.17/1.89	0.70/0.67
GalaxeaVLA (G0.5) [26]	20.23/14.88	18.95/13.00	28.25/20.42	44.12/32.25	8.61/7.33	1.73/1.58
<i>WAM-based</i>						
AHA-WAM [8]	4.82/2.39	5.79/3.00	5.86/2.42	8.61/2.67	2.97/2.78	0.88/0.83
GigaWorld-Policy-0 [48]	6.20/3.27	5.35/3.00	6.15/1.83	15.51/8.92	3.46/2.22	0.54/0.50
X-WAM [13]	7.69/3.83	7.39/3.00	6.72/1.83	17.47/9.08	6.32/4.67	0.57/0.25
<i>Pure VLA with UMI pre-training</i>						
Hy-Embodied-0.5-VLA [52]	13.07/8.80	11.78/8.50	13.81/8.00	25.74/14.92	13.37/12.11	0.65/0.58
Xiaomi-Robotics-1 [39]	20.07/13.93	23.55/17.00	26.69/18.83	38.39/23.67	7.81/6.56	3.94/3.58
<i>Pure VLA without UMI pre-training</i>						
VLAct [37]	10.66/7.60	9.54/6.50	20.62/15.25	20.12/13.67	0.66/0.56	2.37/2.25
InternVLA-A1.5 [28]	11.15/7.14	10.36/7.00	15.23/10.17	23.80/13.75	4.93/3.56	1.43/1.42
$\pi_{0.5}$ [15]	11.41/6.91	13.38/8.00	12.40/5.50	23.54/14.67	5.78/4.56	1.98/1.67
Spatial Forcing [22]	12.38/8.04	14.12/ 9.50	17.33/10.58	23.26/14.58	5.43/4.11	1.78/1.58
Meituan-Robotics-0-VLA	15.27/10.03	14.93/9.00	19.51/10.00	30.96/22.00	8.17/6.67	2.50/2.80

6.1.2 RoboDojo (Waiting for the best result.)

RoboDojo [10] evaluates generalist manipulation policies through a common protocol spanning simulation and real-world settings. Its simulation track contains 42 bimanual tasks organized into five capability dimensions: Generalization, Precision, Long-Horizon, Memory, and Open-vocabulary instruction following. Following the official protocol, we evaluate Meituan-Robotics-0-VLA, a simple VLA without the auxiliary harness or RL-based post-training, and report its Success Rate and Score in Table 4.

Our model achieves an average Score of 15.27 and an SR of 10.03%, ranking first among VLAs and third overall. Without UMI pre-training, explicit memory, or reasoning augmentation, it outperforms all evaluated WAM policies and the UMI-pretrained Hy-Embodied-0.5-VLA. Its Long-Horizon Score of 30.96 suggests that our pre-training recipe and training strategy *improve local decision reliability and limit error accumulation*, while its Memory Score of 8.17 indicates that *strong visual and task priors can support memory-like behavior when historical cues remain visible*. Genuinely ambiguous states, however, still require explicit memory.

Despite these gains, the uniformly low Open scores show that task-level generalization remains difficult for current VLA methods. Large-scale UMI pretraining can partially alleviate this limitation, but tasks that are poorly represented during pretraining remain challenging. For example, on Align_blocks task, we observe that models often default to grasping the box rather than executing the alignment action. This behavior suggests that *current VLAs remain overfit to downstream action patterns*, making task-level generalization a central challenge for future VLA research.

6.2 Real-Robot Evaluation

6.2.1 Evaluation Setup

Robot platform and protocol. We evaluate MR0 on three real-robot tasks using a dual-arm AgileX PiPER system: letter-block placement, garment folding, and network cable insertion. Each model is evaluated for 30 trials per task without human intervention. MR0 and $\pi_{0.5}$ use the same post-training data, observation and action representations, inference settings, and initial task configurations.

Tasks and metrics. We report the success rate (SR) and a normalized task score for each task. The success rate for task τ is



Figure 8: **Network cable insertion.** Head-camera views from a single human-in-the-loop episode, showing the three stages that define the task progress metric $\bar{m}$. The right arm approaches and grasps the cable from the table (*approach*, *grasp*); the cable is then transferred to the left arm, which closes on it before the right arm releases (*handover*, *release*); the left arm finally aligns and inserts the connector into the network port (*insert*). Each stage is a distinct failure mode: localizing a thin, deformable connector against a low-contrast background, coordinating a stable bimanual regrasp, and meeting the tight tolerance of the port. In this episode the operator took over during all three.

$$\text{SR}_\tau = \frac{N_{\text{success},\tau}}{N_{\text{trials},\tau}}. \quad (29)$$

For letter-block placement, seven blocks labeled with the letters in “MEITUAN” are initialized at random positions. A trial is successful when all seven blocks are arranged in the correct order inside the continuous rectangular groove. The task requires accurate grasping, ordering, and placement, and an incorrect placement is usually irreversible. We therefore define the raw task progress as the average number of consecutively correct letters before the first placement error.

For garment folding, the robot must first arrange a crumpled garment and then fold it through coordinated bimanual manipulation. Intermediate errors, such as dropping part of the garment, may still be corrected. A trial is successful when the robot completes the task within 120 seconds. Evaluation continues for up to 300 seconds. A trial completed after 120 seconds is counted as unsuccessful, but its actual completion time is retained. A trial that does not finish within 300 seconds is assigned a completion time of 300 seconds.

Network cable insertion contains three stages (fig. 8): grasping the cable from the table, transferring it from one arm to the other, and aligning and inserting it into the network port. The task requires accurate visual localization, bimanual coordination, and precise insertion control. A trial is successful when all three stages are completed. The raw task progress is the average number of completed stages. We do not use a time metric for letter-block placement or network cable insertion.

Let $\bar{k}$ denote the average number of consecutively correct letters, T the average garment-folding completion time, and $\bar{m}$ the average number of completed stages in network cable insertion. We normalize the three task scores to $[0, 1]$ as

$$S_{\text{block}} = \frac{\bar{k}}{7}, \quad S_{\text{fold}} = \text{clip}\left(\frac{300 - T}{210}, 0, 1\right), \quad S_{\text{network}} = \frac{\bar{m}}{3}. \quad (30)$$

For garment folding, an average completion time of 90 seconds or less gives a Score of 1, while an average completion time of 300 seconds gives a Score of 0. We also report the macro-average SR and Score over the three tasks. SR measures complete task execution, while Score captures partial progress and execution time.

Post-training data. Table 5 summarizes the data used for real-robot post-training. We collect HIL data over four rounds for each task. Each round contains 50 trajectories for letter-block placement, 100 for garment folding, and 200 for network cable insertion. Valid Starts denotes the number of training samples retained after static-interval trimming and control-boundary filtering.

Table 5: Real robot post-training data. Valid-start counts are approximate.

Task	Teleop Traj.	HIL Rounds	Total HIL	Valid Starts
Letter-block placement	500	4	200	~900K
Garment folding	400	4	400	~1.1M
Network cable insertion	200	4	800	~1.0M

6.2.2 Main VLA Comparison

Table 6 compares MR0 with $\pi_{0.5}$. Both models are post-trained and evaluated under the same conditions. MR0 obtains a macro-average SR of 0.86 and a macro-average Score of 0.93. The corresponding results for $\pi_{0.5}$ are 0.82 and 0.86.

On letter-block placement, both models obtain an SR of 0.83. MR0 achieves a higher Score of 0.96, compared with 0.90 for $\pi_{0.5}$. MR0 therefore places more letters correctly before making an error. On garment folding, MR0 improves SR from 0.80 to 0.84 and Score from 0.91 to 0.93. Its average completion time is 117 seconds, compared with 133 seconds for $\pi_{0.5}$. On network cable insertion, MR0 improves SR from 0.83 to 0.90 and Score from 0.78 to 0.89.

Table 6: Real robot evaluation results. Both SR and Score range from 0 to 1. Avg. denotes the macro-average over the three tasks.

Model	Letter-block Placement		Garment Folding		Network Cable Insertion		Avg. SR $\uparrow$	Avg. Score $\uparrow$
	SR $\uparrow$	Score $\uparrow$	SR $\uparrow$	Score $\uparrow$	SR $\uparrow$	Score $\uparrow$		
$\pi_{0.5}$	0.83	0.90	0.80	0.91	0.83	0.78	0.82	0.86
MR0	0.83	0.96	0.83	0.93	0.90	0.89	0.86	0.93

6.2.3 Effect of HG-Dagger-Style Human Intervention

Table 7 compares training with teleoperation data alone against training with both teleoperation and HIL data. Both variants start from the same mid-training checkpoint and use the complete teleoperation dataset. They differ only in whether the accumulated HIL data are included.

Training with teleoperation data alone gives a macro-average SR of 0.08 and a macro-average Score of 0.24. Adding HIL data increases these values to 0.86 and 0.93. The teleoperation-only model does not complete any garment-folding or network cable insertion trial; its nonzero Scores on the other evaluations reflect partial task progress. The HIL data provide corrections for failure cases that are absent from the successful teleoperation demonstrations.

Table 7: Effect of HIL data. Both SR and Score range from 0 to 1.

Training Data	Letter-block Placement		Garment Folding		Network Cable Insertion		Avg. SR $\uparrow$	Avg. Score $\uparrow$
	SR $\uparrow$	Score $\uparrow$	SR $\uparrow$	Score $\uparrow$	SR $\uparrow$	Score $\uparrow$		
Teleop only	0.23	0.41	0.00	0.00	0.00	0.31	0.08	0.24
Teleop + HIL	0.83	0.96	0.83	0.93	0.90	0.89	0.86	0.93

HIL data scaling. We next measure performance after each HIL round. At every round, the model starts from the same mid-training checkpoint and is trained on the complete teleoperation dataset together with all HIL data collected up to that round. The models are trained independently and do not continue from the preceding round. This setup matches the human-gated data-aggregation procedure described in section 4.3.2.

Table 8 shows that both SR and Score increase as more HIL data are added. Letter-block placement reaches an SR of 0.83 and a Score of 0.96 with 200 HIL trajectories. Garment folding reaches an SR of 0.84 and a Score of 0.93 with 400 HIL trajectories. Its average completion time decreases from 180 seconds without HIL data to 117 seconds after the fourth round. Network cable insertion reaches an SR of 0.90 and a Score of 0.89 with 800 HIL trajectories.

Table 8: Performance over four rounds of HIL data collection. Round 0 uses only teleoperation data. HIL is the cumulative number of intervention trajectories. Both SR and Score range from 0 to 1.

Round	Letter-block Placement			Garment Folding			Network Cable Insertion		
	HIL	SR $\uparrow$	Score $\uparrow$	HIL	SR $\uparrow$	Score $\uparrow$	HIL	SR $\uparrow$	Score $\uparrow$
0	0	0.23	0.41	0	0.00	0.00	0	0.00	0.31
1	50	0.60	0.84	100	0.23	0.30	200	0.20	0.48
2	100	0.76	0.87	200	0.27	0.34	400	0.53	0.62
3	150	0.76	0.91	300	0.60	0.84	600	0.70	0.73
4	200	0.83	0.96	400	0.83	0.93	800	0.90	0.89

6.2.4 RECAP Evaluation

To isolate the contribution of RECAP from the preceding supervised stages, this evaluation compares the final human-gated DAGger policy with its RECAP-post-trained counterpart under the same task initialization and rollout protocol. It also reports the effect of the CFG scale separately from the effect of collecting additional HIL data.

6.2.5 Action-Chunk Merging Evaluation

To separate execution-side smoothing from policy quality, this evaluation holds the policy checkpoint and task protocol fixed while comparing direct chunk replacement, online prefix trimming without blending, and the complete action-chunk merging procedure in section 4.4.1. In addition to SR and Score, it reports action discontinuity and peak joint velocity, acceleration, and jerk at chunk boundaries, together with buffer underruns and inference latency at the evaluated inference rates.

6.3 Embodied Agent System Behaviors

Most Vision-Language-Action policies follow an approximately Markovian formulation, predicting actions from the current observation or a bounded window of recent history. Real-world manipulation, however, often exhibits non-Markovian dependencies. Task-relevant information may disappear from the current view after an object is moved or occluded, and an unfinished task may depend on an execution condition that is restored only after a later environmental change. Handling such situations requires maintaining state across observation windows, execution interruptions, and task boundaries.

Our embodied agent system represents this extended temporal context through event memory. The Monitor processes a rolling two-second visual window at 2 Hz and produces temporally grounded descriptions of scene transitions. The memory module consolidates these observations into persistent events containing semantic summaries, task relevance, and supporting keyframes. The Planner retrieves relevant events and combines them with the current task state and latest scene observation, enabling historical state transitions and unresolved execution conditions to participate directly in subsequent planning. The following real-world deployments illustrate cross-task event retrieval and condition-aware task recovery.

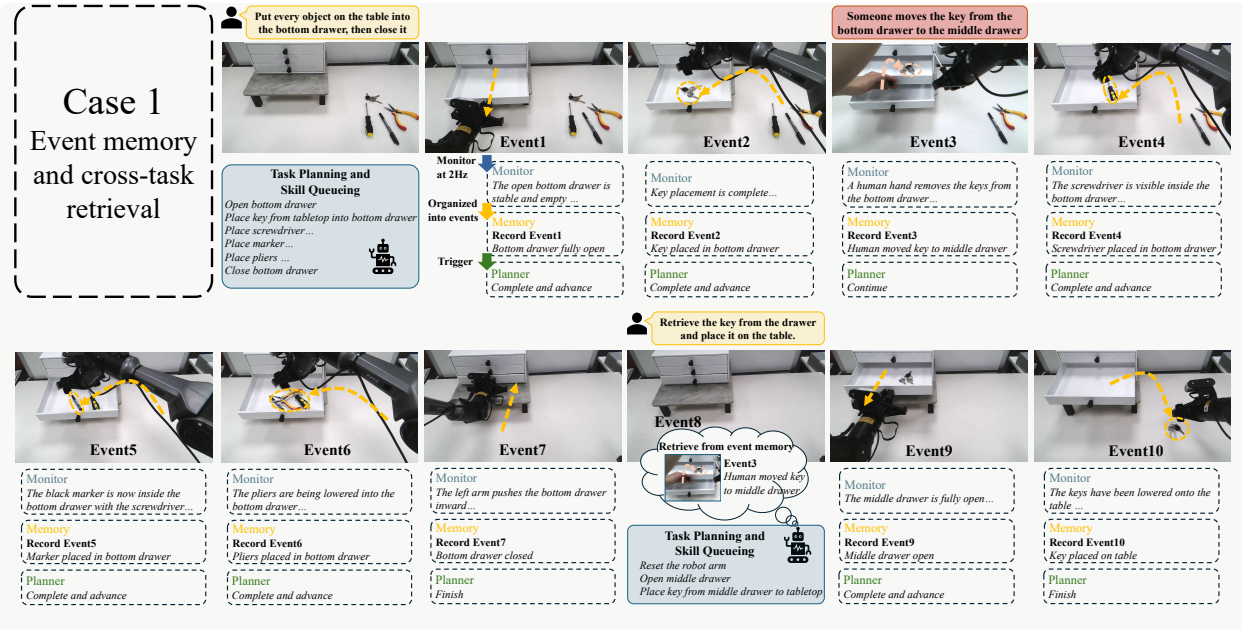


Figure 9: **Event memory and cross-task retrieval.** While the robot is placing the screwdriver into the bottom drawer, a person moves the previously placed key from the bottom drawer to the middle drawer. The intervention is stored as an event together with its visual evidence, while the active screwdriver-placement subtask continues. When the user later requests the key, the stored event and archived keyframes provide the historical evidence required to recover its last observed location and generate a new manipulation plan.

Event memory and cross-task retrieval. As shown in Fig. 9, the agent is instructed to place all objects on the table into the bottom drawer and close it. The Planner decomposes the instruction into drawer-opening, object-placement, and drawer-closing skills, while the Monitor and memory module track the resulting scene transitions.

After the key has been placed into the bottom drawer, the robot advances to the screwdriver-placement subtask. At Event 3, a person removes the key from the bottom drawer and transfers it to the middle drawer. The Monitor captures the intervention, and the memory module records the resulting location change together with the corresponding keyframes.

Because the screwdriver and target drawer remain unaffected, the active subtask remains executable. The Planner therefore continues execution, while Event 3 is retained in persistent memory. The robot completes the remaining object placements and closes the bottom drawer.

After the initial goal is completed, the user asks the robot to retrieve the key from the drawer and place it on the table. The current observation shows a closed cabinet and an empty table, and therefore does not reveal the key location. The new instruction triggers retrieval of task-relevant history from event memory. Event 3 is selected together with its archived keyframes, which indicate that the key was last moved to the middle drawer. The Planner combines this historical evidence with the latest scene observation and generates a skill sequence that opens the middle drawer, retrieves the key, and places it on the table.

This case demonstrates planning under partial observability using persistent event memory. The event summary preserves the semantic state transition, while the archived keyframes retain the corresponding visual evidence. Together, they recover scene information that is no longer directly observable and make it available to planning across task boundaries.

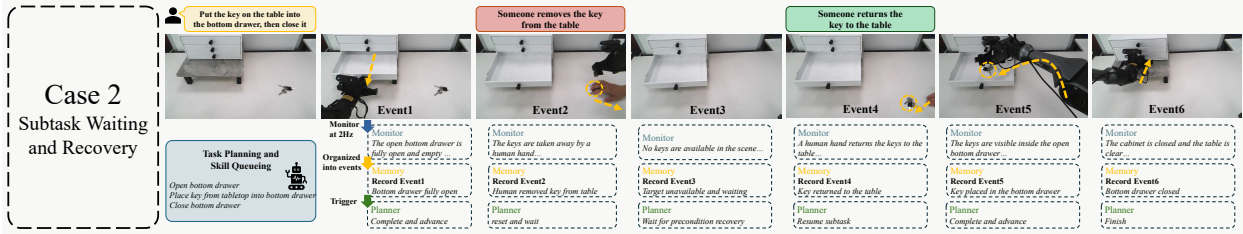


Figure 10: **Subtask waiting and condition recovery.** A person removes the key while the key-placement subtask is active, making the intended action temporarily infeasible. The agent preserves the unfinished objective and continues monitoring the unresolved execution condition. When the key returns, the restored condition triggers replanning from the updated scene, allowing execution to resume and the original instruction to be completed.

Subtask waiting and condition recovery. As shown in Fig. 10, the agent is instructed to place the key into the bottom drawer and close it. The Planner decomposes the instruction into three skills: opening the drawer, placing the key inside, and closing the drawer. After opening the drawer, the robot advances to the key-placement subtask.

Before the grasp is completed, a person removes the key from the table. The Monitor observes that the target object is no longer available, and the memory module associates this change with the execution condition of the active subtask. The resulting event causes the Planner to preserve the pending key-placement objective and transition the system into a waiting state.

Manipulation is suspended while the Monitor continues processing the visual stream. As long as the key remains absent, subsequent observations maintain the unresolved condition in memory without repeatedly invoking the Planner. The pending subtask and the condition required for its execution therefore remain explicitly represented throughout the interruption.

When the key is returned to the table, the Monitor observes that the required object is available again. The memory module associates this transition with the unresolved condition and records a recovery event. The recovery event triggers the Planner to re-ground the pending objective in the latest scene before execution resumes. The robot then grasps the key, places it into the bottom drawer, and closes the drawer, completing the original instruction.

This case demonstrates intention preservation under temporary task infeasibility. The waiting state maintains the pending subtask, continued monitoring tracks its execution condition, and the recovery event identifies when execution becomes feasible again. Re-grounding the objective in the latest scene ensures that resumed execution remains consistent with the environment after the interruption.

Together, the two deployments highlight complementary uses of event memory in embodied task execution. Persistent events preserve task-relevant state transitions that are no longer directly observable and provide historical evidence for subsequent planning. Unresolved events preserve pending intentions and their execution conditions across temporary interruptions. Event summaries, archived keyframes, and current observations therefore provide complementary temporal context for planning over both the present scene and its interaction history.

7 Future Work and Conclusion

8 Contributions

All people are cataloged **alphabetically by first name**. (†) indicates the project leader and (‡) indicates the sponsors.

Contributors:

Caicheng Wang	Jiacong Zhou	Si Liu	Yanbin Zeng	Yuchen Xie [‡]
Chen Zhao	Jie Hu [†]	Shuo Li	Yang Yang	Yuxiang Zheng
Chengpeng Jin	Jiyuan Ru	Tianyi Zhang	Yang Yi	Zerwei Chng
Fengjiao Chen	Linjiang Huang	Xi Zhang	Yanli Tan	Zilong Huang
Guanglu Wan [‡]	Manyuan Zhang	Xin Chen	Yifu Xu	Zipeng Ye
Heqing Yang	Ningchu Liu	Xuezhi Cao	Yimeng Jia	Ziqin Wang
Jia Wang	Shijie Zhou	Xunliang Cai [‡]	Yuchen Tang	Ziyuan Qin

A Real-Robot Setup and Data Preparation

Data interface across teleoperation, HIL, and autonomous rollouts. Our bimanual AgileX PiPER setup contains two human-operated leader arms and two corresponding task-executing follower arms. The operator moves the leader arms, and their joint positions are relayed as target commands to the follower arms. Each trajectory records three camera streams from the task workspace, the observed follower-arm configuration in `observations/qpos`, and the robot command in `action`. During teleoperation and human intervention, `action` is constructed from the leader-arm joint positions and sent to the follower arms; during autonomous execution, it is produced by the policy. Thus, leader-arm state is not a policy observation, but it does determine the supervised action target for human-controlled frames. Both control sources use the same stored camera, state, and action interface.

The demonstration dataset $\mathcal{D}_{\text{demo}}$, collected by teleoperation, contains only successful executions. The tasks and data quantities used in our experiments are specified in section 6.2.1.

Real-robot observation and action representation. The policy receives synchronized images from three RGB cameras: one head camera and two wrist cameras. Each camera captures images at a native resolution of 640×480 . The policy uses a single observation frame and does not include an image history.

For these real-robot experiments, we do not warp the images to a fixed square resolution. We instead use the Smart Resize operation provided by Qwen3.5-VL, which limits the number of visual tokens while preserving the original aspect ratio. Under this task-specific configuration, images are resized to approximately 352×256 before visual encoding. We use the same image processing procedure during post-training and real-robot inference.

The policy state is derived from the observed follower-arm configuration, and the action target from the command sent to the follower arms. Both are converted to the same 20-dimensional end-effector representation. For each arm, this representation contains a 3D Cartesian position, a 6D rotation, and an absolute gripper opening value:

$$\mathbf{x}_t = [\mathbf{p}^L, \mathbf{r}_{6D}^L, g^L, \mathbf{p}^R, \mathbf{r}_{6D}^R, g^R] \in \mathbb{R}^{20}, \quad \mathbf{x}_t \in \{\mathbf{s}_t, \mathbf{a}_t\}. \quad (31)$$

The policy predicts an action chunk containing $H = 50$ control steps:

$$\mathbf{a}_{t:t+H} = [\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+49}] \in \mathbb{R}^{50 \times 20}. \quad (32)$$

Data collection, policy inference, and robot control all operate at 30 Hz. Teleoperation data, HIL data, and autonomous execution therefore use the same control frequency.

Static-interval trimming and valid starts. Real-robot trajectories contain sustained stationary intervals before, during, and after task execution. A transition is considered stationary when consecutive changes in both the observed arm configuration and commanded action remain below their respective thresholds for a minimum run length. We remove the detected intervals before constructing training samples so that hold actions do not dominate the action targets. This is a dataset-specific post-training operation, not a general component of MR0 pre-training or mid-training.

The detector is configured separately for each post-training stage and dataset because they contain different mixtures of slow human corrections, autonomous motion, and pauses. A threshold that is too small leaves idle or near-static intervals in the data, whereas one that is too large removes deliberate slow motion and useful corrective behavior. We therefore calibrate the action- and configuration-change thresholds for the relevant data mixture rather than define one numerical criterion as a general MR0 method.

After trimming and source selection, trajectories are split at episode and control-source boundaries. A frame t is a valid start only when all $H = 50$ target actions from t through $t + 49$ are real, non-tail-padded steps from the same retained segment. Consequently, an action chunk cannot cross a takeover boundary or acquire repeated hold targets through end-of-segment padding. Static-interval trimming and valid-start filtering jointly reduce the bias toward stationary predictions that can otherwise cause the deployed policy to halt. DAGger training uses teleoperation and human-controlled intervention segments, whereas RECAP additionally uses autonomous segments with outcome and advantage labels, as described in section 4.3.3. All retained SFT and DAGger samples receive equal sampling weight.

Post-training implementation details. We use standard image augmentation during post-training: small random crops and rotations are applied to the head-camera view, and color jitter is applied to all three views, while wrist-camera geometry is left unchanged. State and action dimensions are quantile-normalized using their first and 99th percentile statistics; the same training-set statistics are used to unnormalize actions during deployment. These are implementation choices for the real-robot recipe rather than contributions of the method, and their effects are evaluated separately in ??.

B RECAP Implementation Details

Outcome rewards and value targets. Episode-level outcomes are converted into per-timestep rewards:

$$r_t = \begin{cases} 0, & t = T \text{ and the task succeeds,} \\ -T_{\max}, & t = T \text{ and the task fails,} \\ -1, & \text{otherwise,} \end{cases} \quad R_t = \sum_{t'=t}^T r_{t'}, \quad (33)$$

where T is the terminal timestep and $T_{\max}$ is the maximum episode length, which serves both as the terminal failure penalty and as the basis for normalization. Every step incurs unit cost, so the return measures the remaining time to completion, and a failed episode is charged an additional $T_{\max}$. Normalizing by $D = 2T_{\max}$ and clipping gives the scalar target

$$y_t = \text{clip}\left(\frac{R_t}{D}, -1, 0\right) = \begin{cases} \text{clip}\left(-\frac{T-t}{D}, -1, 0\right), & \text{success,} \\ \text{clip}\left(-\frac{T-t+T_{\max}}{D}, -1, 0\right), & \text{failure.} \end{cases} \quad (34)$$

Targets therefore lie in $[-1, 0]$: values closer to zero indicate greater proximity to successful completion, while the terminal penalty separates failed trajectories from successful ones.

Distributional value prediction. The value network predicts a categorical distribution over $B = 201$ atoms instead of directly regressing y_t :

$$p_\phi(V \mid o_t, \ell) \in \Delta^B, \quad z_i = V_{\min} + i \frac{V_{\max} - V_{\min}}{B-1}, \quad i = 0, \dots, B-1, \quad (35)$$

where $V_{\min} = -1$, $V_{\max} = 0$, and the atom spacing is $\Delta z = 0.005$. We project a scalar target onto its two neighboring atoms using

$$b_t = \text{clip}\left(\frac{y_t - V_{\min}}{\Delta z}, 0, B-1\right), \quad l = \lfloor b_t \rfloor, \quad u = \lceil b_t \rceil. \quad (36)$$

When $l < u$, the target masses are $q_l = u - b_t$ and $q_u = b_t - l$. When $l = u$, we set $q_l = 1$; all other entries are zero. The value loss and continuous prediction are

$$\mathcal{L}_V = - \sum_{i=0}^{B-1} q_i \log p_\phi(V = z_i \mid o_t, \ell), \quad V(o_t, \ell) = \sum_{i=0}^{B-1} p_\phi(V = z_i \mid o_t, \ell) z_i. \quad (37)$$

The soft projection preserves local variation in the continuous target while retaining a classification objective.

Value-model architecture. Our value model follows the RECAP implementation in RLinf [35]. It receives the three RGB views, proprioceptive state, and task instruction. A SigLIP2-So400m/14 encoder maps each image to 256 patch tokens projected to 640 dimensions, producing 768 visual tokens across the three views. These tokens are concatenated with the instruction and discretized robot-state tokens and encoded by an 18-layer Gemma 3 270M backbone with hidden dimension 640.

A separate 18-layer value expert with hidden dimension 256 updates a zero-initialized learnable [CLS] token. Through shared attention key-value representations, this token attends to the complete vision-language prefix while the prefix cannot attend back to it. A linear head maps the resulting state representation to the 201 value atoms:

$$h_t^{\text{CLS}} \in \mathbb{R}^{256} \longrightarrow \text{logits}_t = W_V h_t^{\text{CLS}} + b_V \in \mathbb{R}^{201}. \quad (38)$$

The vision encoder and Gemma backbone are initialized from pretrained weights. The value expert and linear head use their default random initialization, and the [CLS] token is initialized to zero. All components are optimized jointly.

Table 9: RECAP value learning, advantage labeling, and policy-training hyperparameters.

Parameter	Setting
Value support $[V_{\min}, V_{\max}]$	$[-1, 0]$
Number of value atoms B	201
Value training steps / batch size	6,000 / 256
Value peak learning rate	5×10^{-5}
Value warmup / final learning rate	500 / 1×10^{-6}
Advantage horizon N / action horizon H	50 / 50
Advantage threshold ϵ	Aggregated-data 70th percentile
Condition-dropout rate p_{drop}	0.3
Policy training steps / batch size	30,000 / 128
Policy peak learning rate	1×10^{-5}
Policy warmup / minimum learning rate	10,000 / 5×10^{-7}

Training configuration. Table 9 lists the configuration used in the current experiments.

References

- [1] AgiBot-World-Contributors et al. AgiBot World Colosseo: A large-scale manipulation platform for scalable and intelligent embodied systems. *arXiv preprint arXiv:2503.06669*, 2025.
- [2] A. Amin, R. Aniceto, A. Balakrishna, et al. $\pi_{0.6}^*$: A vla that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025.
- [3] Marc G. Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning*, pages 449–458, 2017.
- [4] Kaiwen Bi et al. Motus: Motion priors improve vision-language-action models. *arXiv preprint*, 2025.
- [5] Kevin Black, Noah Brown, James Darpinian, Karan Dhariwal, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [6] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [7] Remi Cadene et al. LeRobot: An open-source library for end-to-end robot learning. *arXiv preprint arXiv:2602.22818*, 2026.
- [8] Jisong Cai, Long Ling, Shiwei Chu, Zhongshan Liu, Jiayue Kang, Zhixuan Liang, Wenjie Xu, Yinan Mao, Weinan Zhang, Xiaokang Yang, et al. Aha-wam: Asynchronous horizon-adaptive world-action modeling with observation-guided context routing. *arXiv preprint arXiv:2606.09811*, 2026.
- [9] Zhenyu Cai et al. InternVLA-A1: Unifying understanding and action via latent action modeling. *arXiv preprint*, 2026.
- [10] Tianxing Chen, Yue Chen, Zixuan Li, Junyuan Tang, Kailun Su, Weijie Wan, Baijun Chen, Haoran Lu, Haowen Yan, Honghao Su, et al. RoboDojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies. *arXiv preprint arXiv:2607.04434*, 2026.
- [11] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, Dustin Podell, Tim Dockhorn, Zion English, and Robin Rombach. Scaling rectified flow transformers for high-resolution image synthesis. In *International Conference on Machine Learning*, 2024.
- [12] Kevin Frans, Seohong Park, Pieter Abbeel, and Sergey Levine. Diffusion guidance is a controllable policy improvement operator. *arXiv preprint arXiv:2505.23458*, 2025.
- [13] Jun Guo, Qiwei Li, Peiyan Li, Zilong Chen, Nan Sun, Yifei Su, Heyun Wang, Yuan Zhang, Xinghang Li, and Huaping Liu. Unified 4d world action modeling from video priors with asynchronous denoising. *arXiv preprint arXiv:2604.26694*, 2026.
- [14] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, 2018.

- [15] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [16] Tao Jiang, Tianyuan Yuan, Yicheng Liu, Chenhao Lu, Jianning Cui, Xiao Liu, Shuiqi Cheng, Jiyang Gao, Huazhe Xu, and Hang Zhao. Galaxea Open-World Dataset and G0 Dual-System VLA Model. *arXiv preprint arXiv:2509.00576*, 2025.
- [17] Tao Jiang, Tianyuan Yuan, Yicheng Liu, Chenhao Lu, Jianning Cui, Xiao Liu, Shuiqi Cheng, Jiyang Gao, Huazhe Xu, and Hang Zhao. Galaxea open-world dataset and g0 dual-system vla model. *arXiv preprint arXiv:2509.00576*, 2025.
- [18] Michael Kelly, Chelsea Sidrane, Katherine Driggs-Campbell, and Mykel J. Kochenderfer. HG-Dagger: Interactive imitation learning with human experts. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 8077–8083, 2019.
- [19] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024.
- [20] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. Segment anything. *arXiv:2304.02643*, 2023.
- [21] LejuRobotics. LET: Full-size humanoid robot real-world dataset. [Hugging Face dataset](#), 2025.
- [22] Fuhao Li, Wenxuan Song, Han Zhao, Jingbo Wang, Pengxiang Ding, Donglin Wang, Long Zeng, and Haoang Li. Spatial forcing: Implicit spatial representation alignment for vision-language-action model. In *International Conference on Learning Representations*, volume 2026, pages 132324–132345, 2026.
- [23] Ruicheng Li et al. LingBot-VA: A vision-action model for robotic manipulation. *arXiv preprint*, 2026.
- [24] Tianhong Li and Kaiming He. Back to basics: Let denoising generative models denoise. *arXiv preprint arXiv:2511.13720*, 2025.
- [25] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.
- [26] Yicheng Liu, Zibin Dong, Baijun Ye, Tianyuan Yuan, Tao Jiang, Anqi Yang, Shicheng Cao, Haonan Liu, Yue Sun, Zihan Guo, et al. G0. 5: One autoregressive stream for robot reasoning and action. *arXiv preprint arXiv:2608.11739*, 2026.
- [27] Zhiyu Luo et al. Being-H0.7: A vision-language-action model with hierarchical grounding. *arXiv preprint*, 2026.
- [28] Haoxiang Ma, Junhao Cai, Xiaoxu Xu, Hao Li, Yuyin Yang, Yang Tian, Jiafei Cao, Hongrui Zhu, Zherui Qiu, Yuqiang Yang, et al. Internvla-a1. 5: Unifying understanding, latent foresight, and action for compositional generalization. *arXiv preprint arXiv:2607.04988*, 2026.
- [29] Yao Mu, Tianxing Chen, Shijia Peng, Zanxin Chen, Zeyu Chen, Qiangyu Feng, Zhiqian Su, Weiliang Zhou, Guodong Ma, et al. RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.
- [30] NVIDIA, Johan Bjorck, et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [31] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. FAST: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
- [32] Physical Intelligence Team, Shanghai AI Laboratory. InternVLA-A1.5: Unifying understanding, latent foresight, and action for compositional generalization. *arXiv preprint arXiv:2607.04988*, 2026.
- [33] Qwen Team. Qwen-VLA: Unifying vision-language-action modeling across tasks, environments, and robot embodiments. *arXiv preprint arXiv:2605.30280*, 2026.
- [34] Qwen Team. Qwen3.5: Towards native multimodal agents, 2026. <https://qwen.ai/blog?id=qwen3.5>.
- [35] RLinf Team. RECAP: Offline advantage-based policy optimization. RLinf Documentation, 2026. Accessed August 19, 2026.
- [36] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [37] StarVLA Community. Starvla: A lego-like codebase for vision-language-action model developing. *arXiv preprint arXiv:2604.05014*, 2026.

- [38] StarVLA Community. StarVLA: A modular framework for vision-language-action research, 2026. <https://github.com/StarVLA>.
- [39] Xiaomi Robotics Team, Jun Guo, Piaopiao Jin, Jason Li, Peiyan Li, Yingyan Li, Futeng Liu, Wanli Peng, Optimus Qin, Yifei Su, et al. Xiaomi-robotics-1: Scaling vision-language-action models with over 100k hours of real-world trajectories. *arXiv preprint arXiv:2607.15330*, 2026.
- [40] Yang Tian, Yuyin Yang, Yiman Xie, Zetao Cai, et al. InternData-A1: Pioneering high-fidelity synthetic data for pre-training generalist policy. *arXiv preprint arXiv:2511.16651*, 2025.
- [41] Junjie Wu et al. LingBot-VLA: A vision-language-action model for robotic manipulation. *arXiv preprint*, 2026.
- [42] Kun Wu, Chengkai Hou, Jiaming Liu, Zhengping Che, Xiaozhu Ju, Zhuqin Yang, Meng Li, YINUO Zhao, Zhiyuan Xu, Guang Yang, et al. RoboMIND: Benchmark on multi-embodiment intelligence normative data for robot manipulation. *arXiv preprint arXiv:2412.13877*, 2024.
- [43] Shihan Wu et al. RoboCOIN: An open-sourced bimanual robotic data collection for integrated manipulation. *arXiv preprint arXiv:2511.17441*, 2025.
- [44] X-VLA Team. X-VLA: Soft-prompted transformer as a scalable cross-embodiment vision-language-action model. *arXiv preprint arXiv:2504.20946*, 2026.
- [45] Heqing Yang, Yang Yi, Liyao Wang, Linqing Zhong, Donglin Yang, Ruipu Wu, Zitong Bai, Fengjiao Chen, Manyuan Zhang, Linjiang Huang, and Si Liu. EVA-Client: A unified data collection, inference, and deployment framework for embodied policies on real robots. *arXiv preprint arXiv:2607.02646*, 2026.
- [46] Jianwei Yang, Hao Zhang, Feng Li, Xuayan Zou, Chunyuan Li, and Jianfeng Gao. Set-of-Mark prompting unleashes extraordinary visual grounding in GPT-4V. *arXiv preprint arXiv:2310.11441*, 2023.
- [47] Xingcheng Yang et al. ABot-M0: A vision-language-action model with action manifold learning for bimanual robotic manipulation. *arXiv preprint arXiv:2602.11236*, 2026.
- [48] Angen Ye, Boyuan Wang, Chaojun Ni, Guan Huang, Guosheng Zhao, Hao Li, Hengtao Li, Jie Li, Jindi Lv, Jingyu Liu, et al. Gigaworld-policy: An efficient action-centered world-action model. *arXiv preprint arXiv:2603.17240*, 2026.
- [49] Chenghao Yin et al. Genie Sim 3.0: A high-fidelity comprehensive simulation platform for humanoid robot. *arXiv preprint arXiv:2601.02078*, 2026.
- [50] Checheng Yu, Chonghao Sima, Gangcheng Jiang, Hai Zhang, Haoguang Mai, Hongyang Li, Huijie Wang, Jin Chen, Kaiyang Wu, Li Chen, Lirui Zhao, Modi Shi, Ping Luo, Qingwen Bu, Shijia Peng, Tianyu Li, and Yibo Yuan. χ_0 : Resource-aware robust manipulation via taming distributional inconsistencies. *arXiv preprint arXiv:2602.09021*, 2026.
- [51] Ye Yuan et al. Fast-WAM: World-aware manipulation with fast action generation. *arXiv preprint*, 2026.
- [52] He Zhang, Lingzhu Xiang, Haitao Lin, Zeyu Huang, Minghui Wang, Dingyan Zhong, Yubo Dong, Yihao Wu, Yongming Rao, Dongsheng Zhang, et al. Hy-embodied-0.5-vla: From vision-language-action models to a real-world robot learning stack. *arXiv preprint arXiv:2606.14409*, 2026.
- [53] Yi Zhou, Connelly Barnes, Jingwan Lu, Jimei Yang, and Hao Li. On the continuity of rotation representations in neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5745–5753, 2019.